Authoring Guide Set for Digital Buildings, Materials and Furniture (IFC)

Version 2.5.0, 2026-10-30. Application-neutral guides and tools for outside organisations and their agents.

Authoring Guide Set for IFC Buildings, Materials and Furniture
- Core and profile
- For agents
- The five guides
- Environment quality and remote work
- Submissions
- How rules are written
- Reading order
- Words used in a fixed sense
- Hand-over files
- What this set does not cover

AGENTS.md
- Start here
- Commands
- Working rules
- Layout

Guide 1: Builder's and Architect's Guide
- 1. Vocabulary
- 2. What the profile gives you
- 3. File and schema
- 4. Units, axes, origin and levels
- 5. Rooms and zones
- 6. Walls, doors, windows and openings
- 7. Floors, roofs, ceilings and slab openings
- 8. Circulation, stairs, ramps, lifts, rails
- 9. Names, identifiers, property sets and classification
- 10. Zones and reserved volumes
- 11. Optional extension: lighting fixtures
- 12. Optional extension: signboards
- 13. Geometry hygiene
- 14. Level of detail and budgets

- 15. What to avoid
- 16. Optional extension: terrain context
- 17. Terrain context: the rules

Guide 2: Interior Decorator's Guide

- 1. How consumers decide what a surface looks like
- 2. Colour rules
- 3. Floors, ceilings, wall coverings, accents and trims
- 4. Texture sets
- 5. The finish schedule
- 6. Palettes
- 7. Delivering a reusable material library
- 8. Defects commonly seen
- Exported appearance review

Guide 3: Furniture and Fixtures Guide

- 1. What consumers do with an item
- 2. Classes and categories
- 3. Item library format
- 4. Dimensions, origin and pivot
- 5. Placement points, anchors and where an item may be placed
- 6. Collision and footprint
- 7. Polygon and texture budgets
- 8. Variants and instancing
- 9. Fixtures
- 10. What to avoid

Guide 4: Delivery and Acceptance Guide

- 1. What you send
- 2. Packaging the work
- 3. The manifest
- 4. Versions
- 5. Acceptance checks
- 6. Pre-submission checklist
- 7. How a receiving party accepts, returns or refuses
- 8. Submitting a delivery as a pull request

Guide 5: Worked example

- 1. The profiles
- 2. What is modelled
- 3. Step by step
- 4. Things to try
- 5. What the example leaves out
- 6. Larger examples

How submissions and ingest work

- The path of a delivery
- What a delivery folder holds
- Proofs
- Limits
- The automatic check
- Ingest
- For agents

Environment review and asynchronous handoff

- 1. Agree the work and pin its inputs
- 2. Check construction and usable space before multiplying media

- 3. Review material authoring and exported appearance separately
- 4. Prove the lighting and renderer contract
- 5. Package evidence that another contributor can continue
- 6. Resume and improve

Repair loop

- The loop
- Report fields
- Stop conditions
- Never do these to get a pass
- Worked mini-loop

Rules index

Authoring Guide Set for IFC Buildings, Materials and Furniture

Version 2.5.0 (2026-10-30). Application-neutral guides, rules, checkers and examples for people and software agents who create digital buildings, finishes, wall coverings and furniture in the IFC building-data format, so that their work can be used by many different programs. Start with ABOUT.md: what this is, who it is for, and how a consumer adopts it.

In these guides **the consumer** is whatever program or person receives your delivery (a viewer, a simulator, a game runtime, a BIM tool, an archive). **A profile** is the small JSON file a consumer publishes with its numbers and conventions; if you have none, use `profiles/default.json`. Everything else is in these guides.

Core and profile

- **Core rules** say what is good practice for any consumer and what goes wrong for any consumer if you ignore it.
- **Profile parameters** hold everything numeric or consumer-specific: size and triangle limits, identifier lengths, clearances, property names, whether room identifiers are mandatory, and which optional extensions apply. See 07-profile-parameters.md.
- **Extensions** (`walkability`, `luminaires`, `signboards`, `furnitureCollision`, `terrain`) are optional rule groups a profile switches on.

Check anything against a profile: `python3 tools/check_all.py PATH --profile consumer.json --json`. Reports name the profile used.

For agents

The set is also written to be read and run by software agents of any kind. Static files only; no service is needed.

File	Use
AGENTS.md	short entry point: what the set is, commands, working rules
`llms.txt`, `llms-full.txt`	index of the set, and all content in marked chunks
`skills/`	six skills (orient, author-building, decorate-interior, author-furniture-library, accept-and-package-delivery, submit-delivery), each with a short `SKILL.md` and on-demand `references/`
`rules.json`	all rules with level, applies_to, parameters, extension, how to check, fix hint, examples
REPAIR-LOOP.md	run, read findings, fix, rerun until no REQUIRED finding remains
`profiles/`	the profile schema, the default profile, and an example of a strict consumer profile
`chunks/`, `MANIFEST.json`	the guides in pieces of about 1,500 tokens or less; every file with purpose and tags
`schemas/`	JSON Schemas: profile, rules, checker report, delivery manifest, submission block, proof manifest, catalogue, material library, furniture item, building summary, finish schedule

The five guides

#	File	Who it is for	What it settles
1	01-builders-and-architects-guide.md	architects, builders, modellers	file and schema, units, axes, levels, rooms, walls, floors, stairs, doors, openings, identifiers, property sets, geometry hygiene, optional lighting and signs
2	02-interior-decorators-guide.md	decorators, material and texture authors	finishes, coverings, floors and ceilings, colour keys, finish schedule, texture sets, material libraries
3	03-furniture-and-fixtures-guide.md	furniture and fixture designers	item library format, pivots, dimensions, anchors, clearances, collision, variants, instancing
4	04-delivery-and-acceptance-guide.md	everyone who sends or receives	package layout, manifest with hashes, versions, provenance and licences, checklist, acceptance checks, outcomes
5	05-worked-example.md	everyone	one room with a door, a window, a table, a wall covering and a light, with a generator script and checker

Environment quality and remote work

Follow Environment review and asynchronous handoff for source-to-consumer visual checks, all-room circulation, materials/UVs, lighting, revision invalidation and resumable contributor evidence. Start a handoff with `templates/environment-handoff.template.json`. Technical validation and visual acceptance remain separate decisions.

Submissions

The same repository takes outside deliveries as pull requests from forks: fork it, build the delivery in a branch of your fork under `deliveries/<slug>/`, open a pull request against `main`. Acceptance is a merge to `main`; `tools/ingest.py` then turns the merged delivery into component assets and a neutral catalogue. See 08-submissions-and-ingest.md and CONTRIBUTING.md. Deliveries are not part of the guide set and are not in its hand-over files.

Also: 06-rules-index.md (every rule in one table, with counts), 07-profile-parameters.md, 08-submissions-and-ingest.md, templates/ (starting files), tools/ (scripts), CHANGELOG.md, VERSIONING.md, CONTRIBUTING.md, LICENSE.md.

How rules are written

> **B-12 · REQUIRED** - The statement.
>
> **B-90 · REQUIRED · luminaires** - A statement that applies only in the named extension.

- **REQUIRED** means that without it a consumer is likely to refuse the file, lose the element, or read it wrongly, or (guide 4) that the delivery process requires it. A delivery that breaks a REQUIRED rule is sent back.
- **RECOMMENDED** means good practice that prevents a defect often seen or keeps work portable. A delivery that ignores it is accepted if everything REQUIRED passes, but the reviewer may ask for revision.

- The level shown is the **default**. A profile can raise, lower or switch off any rule (`ruleLevels`); `rules.json` and the checkers use the profile's level.
- Numbers are never in a rule's text as a requirement: they are profile parameters, written in `backticks` (`limits.maxTriangles`).

Identifier letters: **B** builders and architects, **D** decorators, **F** furniture and fixtures, **X** delivery and acceptance. Numbers are never reused; a retired rule keeps its number.

Reading order

1. ABOUT.md, then the vocabulary and profile sections of guide 1 (sections 1 and 2).
2. The guide for your role (1, 2 or 3).
3. Guide 4 before you package anything.
4. Guide 5 once, even if you only make furniture.

Words used in a fixed sense

Word	Meaning here
consumer	the program or person that receives and uses your delivery
profile	a consumer's published parameters (`profiles/profile.schema.json`)
delivery	everything you submit for one assignment (files, manifest, notes)
level	one horizontal floor of a building (an `IfcBuildingStorey`)
room	a named area with a floor, modelled as an `IfcSpace` (the IFC entity name; it always means "room or zone")
zone	a named volume that is data only (`IfcSpatialZone`)
volume	any three-dimensional extent
finish	the look of a surface (colour, texture, gloss)
item	one piece of furniture, fixture or accent
declaration property	a named property this set asks you to write; a profile can rename it
extension	an optional group of rules a profile switches on

Hand-over files

dist/ holds the single-file versions:

File	Content
`builders-guide-2.5.0.pdf`	all guides, the worked example, the submissions page and the rules index in one document
`builders-guide-2.5.0.html`	the same, as one web page
`builders-guide-2.5.0.zip`	the whole set, with `MANIFEST.sha256`
`SHA256SUMS.txt`, `builders-guide-2.5.0.zip.sha256`	SHA-256 of the three files above

`MANIFEST.sha256` lists the SHA-256 of every other file in the set (`sha256sum -c MANIFEST.sha256`); `MANIFEST.json` describes the same files with purpose and tags. `tools/build_dist.py` rebuilds all of this.

What this set does not cover

- Structural, mechanical or electrical design content, and any simulation of how things move or work.
- Terrain. Ground relief is not carried in IFC here; a profile may say how it is supplied.
- Commercial terms between sender and receiver.

AGENTS.md

This folder is an application-neutral guide set for making IFC buildings, finishes (materials, wall coverings) and furniture that many different consumers (viewers, simulators, game runtimes, BIM tools, archives) can use. It is meant to be read and run by agents; people can read the PDF or HTML in `dist/`. Everything is static files and Python scripts: no service, no network.

Start here

1. Read `skills/orient/SKILL.md` (about 70 lines). It tells you which skill to load.
2. Load only the skill you need: `skills/author-building`, `skills/decorate-interior`, `skills/author-furniture-library`, `skills/accept-and-package-delivery`, `skills/submit-delivery`. Each skill points to small files under its `references/` for detail.
3. Rules are in `rules.json` (214, stable ids like `B-41`). Each has a default `level`, `statement`, `rationale`, `parameters`, `extension`, `how_to_check`, `fix_hint`, `examples`, `related`.
4. Consumer-specific numbers are in a **profile** (`profiles/profile.schema.json`; default `profiles/default.json`). Build and check against the consumer's profile when you have one.
5. Find everything else through `llms.txt` (index) or `MANIFEST.json` (every file with purpose and tags).

Review rendered environment quality and remote handoffs with `09-environment-review-and-handoff.md`; use `templates/environment-handoff.template.json`.

Commands

```
pip install -r tools/requirements.txt                        # core; add -r
    tools/requirements-terrain.txt or -preview.txt for those tools
python3 tools/check_all.py PATH --profile PROFILE.json --json --out report.json   # PATH: .ifc file,
    finishes/ folder, items/ folder, or package folder
python3 tools/make_sample.py work/                            # a complete valid example to copy
    from
python3 tools/map_properties.py in.ifc out.ifc --mapping PROFILE.json   # rename declaration
    properties to a consumer's names
python3 tools/site_request.py MASTER.ifc --floor-elevation H --origin E N --out site-request.json
    # only if the profile enables terrain: what the building sends its terrain source
python3 tools/terrain_import.py patch.tif patch.json MASTER.ifc --request site-request.json --out
    OUT.ifc   # only if the profile enables terrain: add the derived terrain patch
python3 tools/make_fixtures.py fixtures/                      # three larger worked buildings, built
    with tools/ifc_author.py
python3 tools/preview.py MASTER.ifc preview/                  # plans and an isometric view, to see
    your own work
python3 tools/run_example.py example-out                      # build, check and package the example
    end to end
python3 tools/selftest.py                                     # prove the checkers still catch every
    deliberate fault
python3 tools/agent_harness.py                                # prove a fresh agent can pass using
    only this set, under two profiles
python3 tools/check_submission.py deliveries/SLUG --json      # form checks of a submitted delivery
    (X-70 to X-78); check_all.py runs them too
python3 tools/build_proofs.py deliveries/SLUG                 # GLB exports, validation report,
    standard renders and proofs/manifest.json (X-79 to X-86)
python3 tools/check_proofs.py deliveries/SLUG --reproduce     # check the proofs, rebuild them and
    compare (needs ifcopenshell, numpy, pillow, shapely)
python3 tools/export_glb.py DELIVERY_OR.ifc --out DIR         # IFC to GLB, whole model and every
    component
python3 tools/validate_glb.py DIR_OR.glb --json               # structure and convention checks of
    GLB files (X-80)
python3 tools/export_glb.py DELIVERY --out DIR --volumes exclude --pivots placement --indexed   #
    scene GLB with per-product pivots, shared meshes, model.placements.json
python3 tools/coplanar.py DIR/model.glb                       # pairs of nodes with coincident faces
    (z-fighting)
python3 tools/walkcheck.py DIR/model.glb --start X,Y,Z --radius 0.30 0.45 --json   # can a body of
    that radius reach every room
python3 tools/check_set.py SET_DIR --expect X.Y.Z             # is a staged copy of this set whole
    and the version it says (X.Y.Z: the version in its CHANGELOG.md)
python3 tools/render.py GLB_DIR --out DIR                     # headless deterministic renders:
    plans, interiors, exteriors, contact sheets
python3 tools/ingest.py deliveries/SLUG --commit HASH --out OUT --dry-run   # maintainers: split a
    MERGED delivery into components and a catalogue
python3 tools/test_submission.py                              # prove the submission checks and
    ingest still behave
python3 tools/test_proofs.py                                  # prove the proof tools catch every
    deliberate fault and reproduce the sample
python3 tools/gen_all.py --write                              # regenerate rules.json, chunks, llms
    files, references, MANIFEST.json
```

Exit codes of every checker: 0 no REQUIRED failure, 1 at least one REQUIRED failure, 2 wrong command line, 3 unreadable input.

Working rules

- Run the checker, read `findings`, fix the deliverable, rerun: `REPAIR-LOOP.md`. Stop after three runs with the same finding id, and report it.
- The profile supplies the numbers that differ between consumers (limits, frame, level offset, identifier rules, build volume, allowed `Occupant` strings, extensions). If one is missing and your work depends on it, ask; do not invent it.

- Never edit a report, a rule level or a tool to get a pass. Never change a delivered version in place.
- Generated files (`rules.json`, `chunks/`, `llms*.txt`, `skills/*/references/`, `MANIFEST.json`, `06-rules-index.md`, `07-profile-parameters.md`) come from the guides, `tools/rules_meta.json` and the profiles; edit those, then run `gen_all.py --write`. `python3 tools/test_sync.py` fails if they differ.
- To contribute a delivery, fork the repository, build it in a branch of your fork under `deliveries/<slug>/` and open a pull request against `main` (`skills/submit-delivery/SKILL.md`). Change only your delivery folder. Acceptance is a merge; ingest runs on merged commits only. Every delivery carries `proofs/` made by `tools/build_proofs.py` (GLB files, validation report, renders); never edit them by hand, rebuild them.
- Describe requirements as what any consumer needs ("the consumer", "the profile"). Do not name products, vendors or projects, and do not claim conformance to or endorsement by any standard or body.

Layout

`skills/` skills; `tools/` scripts and `rules_meta.json`; `profiles/` profile schema, default and example; `schemas/` JSON Schemas; `templates/` starting files; `chunks/` guide sections of about 1,500 tokens or less; `01-05` guides; `REPAIR-LOOP.md`; `ABOUT.md`; `dist/` PDF, HTML, zip and checksums; `deliveries/` submitted deliveries (not part of the set or its hand-over files); `.github/` pull request template and workflow.

Guide 1: Builder's and Architect's Guide

For architects, builders and modellers who deliver buildings, public areas and their fittings as IFC files that many different consumers (viewers, simulators, game runtimes, BIM tools, archives) can open and use.

Rule levels and the profile mechanism are explained in the README. Every number that depends on a consumer is a **profile parameter**, written in `backticks` like `limits.maxTriangles`; its default is in `profiles/default.json` and the full list is in 07-profile-parameters.md.

1. Vocabulary

Term	Meaning
consumer	any program or person that receives your delivery and uses it: a viewer, a simulator, a game runtime, a BIM tool, an archive
profile	the JSON file a consumer publishes with its parameters (limits, identifier rules, optional extensions); the default profile is `profiles/default.json`
master file	the single coordinated IFC file for one placement area
level	one floor of a building: an `IfcBuildingStorey`
room	a named area with a floor, modelled as an `IfcSpace` (the IFC entity name; here it always means "room or zone")
zone	a named volume that is data only: an `IfcSpatialZone`
declaration property	a named property this set asks you to write because consumers read it (for example `RoomId`); a profile can rename them
extension	an optional group of rules a profile switches on when its consumer needs it: walkability, luminaires, signboards, furniture collision, terrain

2. What the profile gives you

Before you start, get the profile of the consumer you build for. If you have none, use `profiles/default.json` and say so in your delivery notes. If a number your work depends on is missing, ask the consumer; do not guess.

Profile item	What it tells you	Parameters
frame	which way +X and +Y point, the height of your local z = 0 in the consumer's frame, whether the file is placed as authored	`frame.horizontalAxes`, `frame.levelOffsetMetres`, `frame.placement`
limits and budgets	hard caps and recommended sizes	`limits.*`, `budgets.*`
identifiers	room identifier length and pattern, assigned entity-number range, allowed occupants	`identifiers.*`, `propertyNames`
geometry	doorway, stair and ramp values	`geometry.*`
volumes	the region to stay inside, regions to keep out, floors the consumer supplies	`buildVolume`, `keepOut`, `baseSurfaces`

Profile item	What it tells you	Parameters
extensions	whether it walks people through rooms, turns fixtures into lights, generates signs, simplifies furniture, supplies terrain	`extensions.*`
rule levels	rules it makes stricter or switches off	`ruleLevels`

3. File and schema

B-01 · REQUIRED - Deliver an IFC schema edition the profile accepts (`schema.allowed`, default IFC4). Other editions may open in some tools, but files in an unlisted edition are not portable to that consumer.

B-02 · REQUIRED - Declare the length unit as metres (an `IfcSIUnit` `LENGTHUNIT` `METRE`, or a prefixed or conversion-based unit that resolves to metres) and model in it. A missing or misread unit rescales the whole building.

B-02a · RECOMMENDED - Prefer plain metres (a `METRE` unit with no prefix) over a prefixed or conversion-based unit, which some tools mishandle.

B-03 · RECOMMENDED - Deliver one coordinated master file per placement area, and send per-building files only as alternates. Several overlapping files for one area leave the consumer guessing which one is authoritative.

B-04 · REQUIRED - Keep the file plain ASCII: write every non-ASCII character with the format's escape (`\X2\00B7\X0\` for a middle dot, `\X2\2014\X0\` for a long dash). Raw bytes above 127 are read differently by different tools.

B-05 · REQUIRED - Stay within the profile's hard limits: `limits.maxFileMegabytes`, `limits.maxVertices` and `limits.maxTriangles`. A file over a consumer's cap is refused or truncated.

B-06 · REQUIRED - Use geometry that every common reader can display: swept solids (`IfcExtrudedAreaSolid` with rectangle, circle, arbitrary closed or arbitrary-with-voids profiles), faceted breps or polygonal face sets, and mapped items. Boolean-heavy breps, swept disks and unusual profiles are dropped by many readers without a message; `check_ifc.py` reports products that have a representation but yield no triangles.

B-07 · RECOMMENDED - Make the export deterministic: the same inputs give a byte-identical file (fixed timestamps, stable ordering, `GlobalId` values derived from stable keys). Consumers and reviewers compare revisions.

B-08 · RECOMMENDED - Write each STEP entity on a single line when the profile asks (`file.singleLineEntities`). Line-oriented tools, diff tools and simple readers handle such files; wrapped entities are valid but can be missed. A consumer that depends on it raises this rule to required.

B-09 · RECOMMENDED - Keep the file below the profile's recommended size, `budgets.recommendedFileMegabytes`. Large files load slowly and are hard to review.

4. Units, axes, origin and levels

B-10 · REQUIRED - Use a right-handed frame with Z up and the horizontal axes the profile states (`frame.horizontalAxes`), in metres. Models authored with another up axis come in turned over.

B-11 · RECOMMENDED - Author in the placement the profile expects (`frame.placement`): with `final`, local z = 0 is the finished floor of the lowest level and the consumer applies no offset or rotation, so author in the final placement frame. If you cannot, state the offset in the frame note (B-12).

B-12 · RECOMMENDED - State your frame in the file: a property set on the `IfcProject` with a text property `FrameNote` ("metres; +X forward; +Y left; +Z up; local z = 0 at the finished floor of the lowest level") and a number `LevelOffsetMetres`. A written note lets a reviewer or another tool verify the frame.

B-13 · REQUIRED - Create one `IfcBuildingStorey` for every level of every building, with a `Name` and a numeric `Elevation` equal to the level's finished-floor height in your file's frame, and make every element belong to one: contained (`IfcRelContainedInSpatialStructure`), aggregated (`IfcRelAggregates`) or, for zones, referenced (`IfcRelReferencedInSpatialStructure`). Elements without a level are dropped whenever a tool works level by level.

B-14 · RECOMMENDED - Use the full hierarchy `IfcProject` > `IfcSite` > `IfcBuilding` > `IfcBuildingStorey`, one `IfcBuilding` per building, and level names that start with the building name (`Clinic - ground / +0 m`).

B-15 · RECOMMENDED - Keep floor-to-floor heights regular so that stairs, lifts and openings line up between levels; make tall halls an exception you can name.

B-142 · RECOMMENDED - Put nothing below the lowest finished floor except slabs, footings, piles, walls, stairs, ramps, openings, the terrain element and reservation volumes. Foundations, retaining walls and a stair that goes down into the ground are ordinary on a slope; a door, window, room or piece of furniture below the lowest floor means the origin was put at the wrong height.

5. Rooms and zones

A room is the unit a consumer identifies, lists, lights, signs, and places people in.

B-20 · REQUIRED - Model every room as an `IfcSpace` (internal, `ELEMENT`) whose geometry is an upward extrusion of the room's floor outline: the lowest plane of the solid is the floor and the top is the underside of the ceiling or slab above. Tools derive the floor and area from that lowest plane, so the floor must be one flat horizontal plane.

B-21 · REQUIRED - Give every `IfcSpace` geometry. A room without geometry cannot be located, measured or shown.

B-22 · REQUIRED - Give every room that has a `RoomId` a value that is unique, and write it as an `IfcPropertySingleValue` inside an `IfcPropertySet` linked by `IfcRelDefinesByProperties`. Duplicate identifiers make every reference to a room ambiguous.

B-23 · REQUIRED - Make room identifiers meet the profile: at most `identifiers.roomIdMaxLength` characters and matching `identifiers.roomIdPattern`. Consumers print identifiers on labels and store them in fixed-width fields.

B-24 · RECOMMENDED - Give every room an `Occupant` (text): the organisation or use it belongs to. When the profile lists allowed values (`identifiers.occupants`) use one of them exactly; consumers classify rooms by this text.

B-25 · RECOMMENDED - Give every room a `Function` (text) describing its use. When the profile names a convention for circulation (`identifiers.circulationPrefix`), start corridors, lobbies, stair halls and streets with it.

B-26 · RECOMMENDED - Give every room a `RoomId`. Tools that find no identifier invent one from the room's name, and that changes when the name changes, which breaks every saved reference. A consumer that needs stable references raises this rule to required (`ruleLevels`).

B-27 · RECOMMENDED - Also supply, when known: `NetAreaSquareMetres` (number), `BaseLevelMetres` (number), `ClearHeightMetres` (number) and `Notes` (text). IFC's standard property set for rooms has related properties (for example `Reference` and `NetPlannedArea`); a profile may map the declaration names to those (`propertyNames`).

B-28 · RECOMMENDED - Give each room a complete, human-readable `Name`; names appear in lists and on labels.

B-29 · RECOMMENDED - Declare streets, paths, plazas and park areas as rooms too, with an identifier and a `Function`, so that they can be referred to, lit and signed. Open floor that belongs to no room has no identity.

B-143 · RECOMMENDED - Model an open-air room (terrace, courtyard, deck, path, plaza) as an `IfcSpace` with `PredefinedType` `EXTERNAL` and the boolean property `Enclosed` set to false, and an internal room with `Enclosed` true or absent. Its volume rises to what covers it, or to a nominal height where it is open to the sky. Consumers that look for enclosed rooms (lighting, walkability, ceilings) skip open-air rooms.

B-30 · RECOMMENDED - Let rooms on one level touch along wall centre lines but not overlap by more than the wall thickness; overlapping floors make "which room is this point in" ambiguous.

B-31 · RECOMMENDED - Draw the room outline at the inside wall faces so the floor area is the usable area.

B-32 · RECOMMENDED - Group rooms into buildings with a `BuildingId` property, and mark long-lived content separately from replaceable content with a `Layer` property (`permanent` or `interim`), so that a building can be swapped without touching its surroundings.

What a consumer typically builds from a room

Place field	Comes from
identifier	`RoomId` (or one invented from the name when missing)
name, function, occupant, notes	`Name`, `Function`, `Occupant`, `Notes`
kind and organisation	rules on `Function`, `Occupant`, `RoomId` and `Name`
level	the level that holds the room
floor and ceiling height	lowest and highest point of the room solid
area	floor triangles; `NetAreaSquareMetres` when given
centre, bounds, floor outline	the room solid and its floor
doors and neighbours	the openings filled by doors (B-41)

Walkability extension: standing points

These two rules apply when the profile enables `extensions.walkability` (a consumer that places people or agents in rooms).

B-33 · REQUIRED · walkability - Leave every enclosed room a floor area where a person can stand: at least `extensions.walkability.standingClearanceMetres` clear of furniture, equipment, rails and transport elements. A room whose whole floor is within that distance of fixed objects has no usable start point.

B-34 · RECOMMENDED · walkability - Keep a free floor circle of `extensions.walkability.freeCircleMetres` in every room, and leave clear aisles from seating to doors.

6. Walls, doors, windows and openings

B-40 · REQUIRED - Model walls as `IfcWall` (`IfcWallStandardCase` is also read), not as generic proxies; consumers select, colour and count by class.

B-40a · RECOMMENDED - Use one element per maximal straight run rather than one per room edge.

B-41 · REQUIRED - Model every door as an `IfcDoor` that fills an `IfcOpeningElement` in the wall, linked with `IfcRelVoidsElement` (wall to opening) and `IfcRelFillsElement` (opening to door). Tools find doorways, count them and work out which rooms they connect from the opening; a door with no opening is a picture on a solid wall.

B-42 · REQUIRED - Keep every doorway passable: its clear height is at least `geometry.doorMinClearHeightMetres`, and nothing solid (a sill, a bar, a glazed panel, furniture) crosses the opening centre line at the heights in `geometry.doorwayProbeHeightsMetres`. A door leaf may stand open or closed in the model, but do not rely on it to block movement.

B-43 · RECOMMENDED - Make doorways at least `geometry.doorMinClearWidthMetres` wide.

B-44 · REQUIRED - Model every window as an `IfcWindow` in an `IfcOpeningElement` linked with the voids and fills relations, so the wall is cut where the window is.

B-45 · RECOMMENDED - Give glass a surface style with opacity below 1 (about 0.4 to 0.6), so that every consumer shows it as see-through.

B-46 · RECOMMENDED - Give walls their real thickness; zero-thickness walls and double walls confuse area, collision and rendering.

B-47 · RECOMMENDED - Make every room reachable through a doorway, an open edge or a stair from a circulation room.

B-48 · RECOMMENDED - Do not put a door in a floor or roof; floor apertures are slab openings (B-53).

7. Floors, roofs, ceilings and slab openings

B-50 · RECOMMENDED - Model every walkable floor, including walkable roofs, as an `IfcSlab` (`FLOOR` or `ROOF`) whose top face is the finished floor. Rooms standing on a floor the consumer supplies (`baseSurfaces`) need no slab of their own. Glass floors are slabs with a glass-like colour and opacity. A consumer that checks standing points or floor contact raises this to required.

B-51 · RECOMMENDED - Do not duplicate a floor the consumer already provides (`baseSurfaces`); two coplanar surfaces flicker.

B-51a · RECOMMENDED - Omit slabs that rest directly on a consumer-supplied floor, or make them very thin.

B-52 · RECOMMENDED - Give slabs a realistic thickness; very thin slabs are fine only for surface layers such as paving.

B-53 · REQUIRED - Cut every hole in a slab (stair wells, shafts, service drops) as an `IfcOpeningElement` linked by `IfcRelVoidsElement`, not as separate floor pieces that merely leave a gap.

B-54 · RECOMMENDED - Model ceilings or roofs. Many consumers draw no ceiling unless there is geometry; a room with nothing above it reads as open-topped. A ceiling is the underside of a slab or an `IfcCovering`.

B-144 · RECOMMENDED - Model a pitched or sloped roof with sloped `IfcSlab` `ROOF` pieces that follow the real pitch, and a wall whose faces lean (a battered wall) as one `IfcWall` with an inclined section, not as flat boxes or steps. Consumers that work out rain, snow, sight lines or shadows read the shape; a stepped approximation gives the wrong answer.

B-55 · RECOMMENDED - Keep your own structure clear of openings in surfaces the consumer supplies (`baseSurfaces`), with the margin the profile states.

8. Circulation, stairs, ramps, lifts, rails

B-60 · REQUIRED - Model each stair flight as an `IfcStairFlight` (a parent `IfcStair` without geometry is fine). Consumers recognise stairs, and simulators build their walking surface, from the flight.

B-61 · RECOMMENDED - Keep risers at or below `geometry.stairRiserLimitMetres`, with a landing and head-room; avoid winding and spiral stairs unless you have checked the consumer reads them.

B-62 · RECOMMENDED - Give every multi-level building at least one real stair. Lifts drawn in IFC are often static solids; a stair is the one way between levels that every consumer understands.

B-63 · RECOMMENDED - Draw lifts as `IfcTransportElement` with `PredefinedType` `ELEVATOR`, with properties such as `Service`, `StopsMetres` and `ShaftWidthMetres`.

B-64 · RECOMMENDED - Keep every walked slope at or below `geometry.rampMaxSlope` (rise over run) except stairs, and give ramps a clear width at least as wide as a doorway. Model ramps as `IfcRamp` with `IfcRampFlight`, or as a sloped `IfcSlab`.

B-65 · RECOMMENDED · walkability - Do not leave low obstacles up to `extensions.walkability.stepHeightMetres` high (floor boxes, square-edged wedges) that a walker steps over unseen; ramp them or make them visible steps.

B-66 · RECOMMENDED - Model rails (`IfcRailing`) with cheap geometry: simple posts and one rail section. Rails with dense tessellation are a common cause of heavy files.

B-67 · RECOMMENDED · walkability - Know which classes the consumer treats as not solid (`extensions.walkability.nonSolidClasses`) and model walk-through areas with those classes or with reservation volumes (B-80); everything else may block movement.

Clearance and size summary

Item	Value	Where it comes from
doorway clear height	at least `geometry.doorMinClearHeightMetres`	B-42
doorway probe heights	`geometry.doorwayProbeHeightsMetres`	B-42
doorway clear width	at least `geometry.doorMinClearWidthMetres`	B-43
stair riser	at most `geometry.stairRiserLimitMetres`	B-61
walked ramp slope	at most `geometry.rampMaxSlope`	B-64
free floor around fixed objects	`extensions.walkability.standingClearanceMetres`	B-33
free floor circle in a room	`extensions.walkability.freeCircleMetres`	B-34

9. Names, identifiers, property sets and classification

B-70 · REQUIRED - Give every product a unique 22-character IFC `GlobalId` that does not change between exports. `GlobalId` is the key tools use to match an element across revisions and across files.

B-71 · RECOMMENDED - Number your IFC entities from the start the profile assigns (`identifiers.entityNumberStart`, when set) so that files from several authors can be merged without clashes.

B-72 · RECOMMENDED - Write declaration properties as text, number or boolean values in `IfcPropertySingleValue` entries of an `IfcPropertySet` linked by `IfcRelDefinesByProperties` on the product itself. Many readers ignore quantity sets, enumerations, tables and properties on types.

B-73 · RECOMMENDED - Use one descriptive property set per topic (`Pset_Declaration_Room`, ...) and put only declaration properties in it.

B-74 · RECOMMENDED - If your organisation uses a classification scheme, attach it with `IfcRelAssociatesClassification`. Many consumers ignore it, so never make it the only place a meaning is written.

B-75 · RECOMMENDED - Keep identifiers stable across revisions: an unchanged room, door or fixture keeps its `RoomId`, `GlobalId` and declaration ids. Replace an element by giving the new one a new id and retiring the old.

B-76 · RECOMMENDED - Give your own identifiers dotted prefixes (`arch.entrance`, `clinic.lobby`), at most 160 characters, no control characters.

10. Zones and reserved volumes

B-80 · RECOMMENDED - Declare a volume that must stay empty or must not be solid (a sight line, a clearance, a service corridor) twice: as an `IfcSpatialZone`, and (so it can be seen) as an `IfcBuildingElementProxy` with `PredefinedType` `PROVISIONFORSPACE`. That is the standard IFC way to mark a reserved provision: consumers that understand it leave the proxy out of physical models; any other proxy is a solid thing.

B-81 · RECOMMENDED - Give every zone a unique identifier (`ZoneId` or, if missing, its `Name`) and its own geometry; a zone without geometry cannot be shown or selected.

B-81a · RECOMMENDED - Always supply `ZoneId`, plus `ZoneClass` (for example `observation-sightline`) and `Requirement` (a sentence saying what must stay clear).

B-82 · RECOMMENDED - Use zones to record viewing intent (a downward sight line from a mezzanine); ordinary glass cannot be one-way, and who may look is a consumer decision, not a file property.

11. Optional extension: lighting fixtures

These rules apply when the profile enables `extensions.luminaires` (a consumer that turns declared fixtures into lights). Property names are those of the profile (`propertyNames`).

B-90 · REQUIRED · luminaires - Model each fixture as an `IfcElementAssembly` that aggregates two `IfcLightFixture` parts: a housing, and an emissive part whose `Name` ends in the profile's suffix (`extensions.luminaires.lensNameSuffix`). The emissive part is the one that glows.

B-91 · REQUIRED · luminaires - Put every property in `extensions.luminaires.requiredProperties` on the assembly, make `LuminaireId` unique, and use a `LightType` from `extensions.luminaires.lightTypes`. A fixture missing a property cannot be turned into a light.

Property	Type	Meaning
`LuminaireId`	text	unique identifier
`LuminaireMount`	text	`pendant`, `ceiling`, `post`, `wall`, free text
`LightType`	text	one of the profile's light types
`LightEnergy`	number	strength in the consumer's units
`LightRangeM`	number	reach in metres
`LightSpotDeg`	number	cone angle in degrees
`LightColor`	text `"r,g,b"`	each 0 to 1
`LightPosition`	text `"x,y,z"`	where the light sits, in the consumer's global frame (B-92)
`LightAim`	text `"x,y,z"`	unit vector the light points along
`LightFadeBeginM`, `LightFadeLengthM`	number	where the light starts to fade, and how long the fade is
`LuminaireTarget`	text, optional	the `RoomId` the fixture serves

B-92 · REQUIRED · luminaires - Write `LightPosition` in the consumer's global frame: the file-local point plus `frame.levelOffsetMetres` (same x and y, z plus the offset); the geometry stays file-local. The emissive part's centre plus the offset must be within `extensions.luminaires.positionToleranceMetres` of the declared point.

B-92a · RECOMMENDED · luminaires - Give the file-local position as well (`LightPositionLocal`) so a reviewer can check the offset was applied once.

B-93 · REQUIRED · luminaires - Make `LightAim` a unit vector (length 1 within 1e-6) in the consumer's frame.

B-94 · RECOMMENDED · luminaires - Give every enclosed room at least one fixture (set `LuminaireTarget`), between the floor and the ceiling, in warm colours, not pointed at glazing.

B-95 · RECOMMENDED · luminaires - Count the lights: every declared fixture may become a real light, and the consumer may cap them per room (`extensions.luminaires.maxPerRoom`).

12. Optional extension: signboards

These rules apply when the profile enables `extensions.signboards` (a consumer that generates signs from declared boards).

B-100 · REQUIRED · signboards - Model a board as an `IfcElementAssembly` carrying every property in `extensions.signboards.requiredProperties`, with a `SignboardKind` from `extensions.signboards.kinds`.

Property	Type	Meaning
`SignboardId`	text	unique identifier
`SignboardKind`	text	one of the profile's kinds (default `entrance_board`, `stop_board`)
`SignboardText`, `SignboardSub`	text	main text, optional second line
`SignboardWidthM`, `SignboardHeightM`	number	board size in metres
`SignboardReadM`	number	distance in metres from which it must be readable
`SignboardPosition`	text `"x,y,z"`	centre, in the consumer's global frame (B-92)
`SignboardNormal`	text `"x,y,z"`	unit vector pointing away from the board's face

B-101 · REQUIRED · signboards - Make the main text fit the board: allow about 0.55 times the capital height of width per character plus a margin of 12 % of the board height on each side; text that cannot be fitted at a capital height of 0.05 m is an error.

B-102 · RECOMMENDED · signboards - Declare a reading distance your letters support: capital height in metres must be at least `pixels * readM / extensions.signboards.displayPixelsPerRadian`, with pixels from `extensions.signboards.minCapitalPixels` (primary text), and the distance within `extensions.signboards.readingDistanceMetres`.

B-103 · RECOMMENDED · signboards - Declare an entrance board on the wall's outer face, not the wall centre plane, and keep lamps and trees away from its face.

13. Geometry hygiene

B-110 · REQUIRED - Keep all geometry inside the profile's build volume (`buildVolume`) and outside its keep-out regions (`keepOut`), when it gives them; check every vertex, not only footprints. The terrain element of the terrain extension (section 16) is context supplied by the consumer's terrain source and is not checked against the build volume.

B-111 · RECOMMENDED - Make solids closed and consistently oriented: no missing faces, no inside-out solids, no zero-area or duplicate faces.

B-112 · RECOMMENDED - Avoid coplanar overlapping faces (two walls sharing a face, a covering lying exactly on a wall, a slab exactly on a floor). Offset thin layers or merge them.

B-113 · RECOMMENDED - Use mapped items (`IfcMappedItem` with `IfcRepresentationMap`) for repeated parts. They keep files small; most consumers expand them to ordinary triangles, so they save file size, not drawing cost.

B-114 · RECOMMENDED - Do not model what cannot be seen or used: hidden interiors, bolts, hinge detail, text as geometry, curves with more than `geometry.maxCurveSegmentsPerCircle` segments per full circle.

B-115 · RECOMMENDED - Do not model terrain, soil or relief in IFC unless the profile says how. Trees and planting may be `IfcGeographicElement` markers; consumers differ on whether they are solid. The terrain extension (section 16) is how a profile says how: with it, the only terrain in the file is the derived patch it describes.

B-116 · RECOMMENDED - Do not model a thin layer at a height where the consumer adds its own surface layer (floor finishes laid on the room floor); two layers at one height flicker.

14. Level of detail and budgets

Triangle count drives load time; the number of distinct materials and objects drives drawing cost. Hard caps are `limits.*` (B-05); recommended sizes are `budgets.*`.

Quantity	Profile parameter	Default
file size, hard	`limits.maxFileMegabytes`	250 MB
vertices, hard	`limits.maxVertices`	10,000,000
triangles, hard	`limits.maxTriangles`	5,000,000
file size, recommended	`budgets.recommendedFileMegabytes`	100 MB
triangles, recommended	`budgets.recommendedTriangles`	1,000,000
distinct (class, colour, opacity) combinations	`budgets.maxDistinctColours`	50

B-120 · RECOMMENDED - Stay under the profile's recommended triangle count, `budgets.recommendedTriangles`. A constrained consumer publishes a much lower figure; build to the profile you were given. Terrain triangles are counted separately (B-138).

B-121 · RECOMMENDED - Spend triangles where the viewer looks: entrances, interiors a person stands in, anything within arm's reach. Distant detail is wasted.

15. What to avoid

- A floor that duplicates one the consumer supplies (B-51).
- Doors without openings, or openings without doors (B-41).
- Anything across a doorway (B-42).
- Rooms without identifiers or with duplicates (B-22, B-26).
- Heavily tessellated rails and balusters (B-66).
- Low steps and ledges nobody can see (B-65, with the walkability extension).
- A proxy used for something that must be walk-through; use a reservation volume (B-80).
- Missing ceilings (B-54).
- Elements outside any level (B-13).
- Geometry readers drop silently (B-06).
- One-way glass promises: ordinary glass is two-way.
- Changing identifiers between revisions (B-75).

- Cutting a pad, a cut or a fill into the terrain element, or modelling water in the file (B-130, B-139).

16. Optional extension: terrain context

Some consumers place a building in a world whose ground comes from their own terrain source. There the terrain stays authoritative in the source. In IFC it is **derived, read-only context**: a terrain element built from a patch the source gives you, used for placement, clash and "is the building floating or buried" checks. The extension applies when `extensions.terrain.enabled` is true; its numbers are in `extensions.terrain.*`.

The work flows like this:

1. **Send** the source three things: the footprint polygon of every ground-contact level, the finished floor elevation of each (height above sea level of its finished floor) and a grading rule (`extensions.terrain.grading`, for example cut and fill slopes no steeper than 1:2). `tools/site_request.py` writes them from the master file.
2. **Receive** a height patch: a GeoTIFF with one float32 band in metres, cut round the building with a margin, at one source level (one pixel size), and a JSON sidecar (`schemas/terrain-patch.schema.json`) with the source package version and hash, origin, pixel size, CRS definition, vertical datum and licence. The source grades the pad and clears vegetation inside the footprint plus a margin (`extensions.terrain.clearingMarginMetres`). Its baseline ground is natural, with no built features; the building is an overlay layer and never part of the terrain.
3. **Import** with `tools/terrain_import.py`: it reduces the samples to a triangulated network within a stated error bound, converts to the file frame and writes the elements the rules below describe.
4. **Check** with `check_ifc.py` and a profile that enables the extension.

Say which frame each number is in:

Where	Frame
sidecar and site request	site frame: x east, y north, z up, metres, centred on the site; heights above mean sea level, with sea level exactly 0
IFC file	file frame: the site frame shifted by the building origin and turned by the building azimuth (the angle from east, counter-clockwise, to the file's +X axis), z measured from the lowest finished floor
the consumer's own runtime	whatever it uses (for example x east, y up, z south); the consumer converts, you never write it

The site frame is a custom azimuthal frame on a sphere, not a registry code. `IfcProjectedCRS.Name` carries the profile's name for it (`extensions.terrain.crsName`) and `Description` carries the definition from the sidecar.

17. Terrain context: the rules

These rules apply when `extensions.terrain.enabled` is true (section 16 explains the flow and the frames).

B-130 · REQUIRED · terrain - Deliver the terrain as exactly one `IfcGeographicElement` with `PredefinedType` `TERRAIN`, contained in the `IfcSite`, whose body is one `IfcTriangulatedFaceSet`. It is derived, read-only context: do not edit its heights, and do not cut pads, cuts, fills or building

features into it. The terrain source does grading and clearing before it cuts the patch, so the world and the model agree.

B-131 · REQUIRED · terrain - Make the terrain a valid heightfield over a rectangular patch: every triangle faces up and has area, no edge belongs to more than two triangles, and the triangles cover the patch rectangle exactly once. Consumers sample the surface by plan position, so an overhang, a hole or an overlap gives two heights or none for one point.

B-132 · REQUIRED · terrain - Make the terrain cover the building: the plan points of the slabs and walls of every ground-contact level, grown by `extensions.terrain.footprintMarginMetres`, lie inside the patch. A patch that stops short of the footprint cannot answer where the ground is under the building.

B-133 · REQUIRED · terrain - Georeference the file with an `IfcMapConversion` from the model context to an `IfcProjectedCRS` named `extensions.terrain.crsName`, in metres, with the local X axis set to the building azimuth (a unit vector, east when the building is not turned), scale 1 and `OrthogonalHeight` equal to the height above sea level of the lowest finished floor (local z = 0). The name need not be a registry code. The profile's `frame.levelOffsetMetres` must be 0, because the floor height is carried here and not by an offset.

B-134 · REQUIRED · terrain - Set `IfcSite.RefElevation` to the terrain height at the building origin above sea level, within `extensions.terrain.siteElevationToleranceMetres` of the terrain element at that point. The site elevation and the terrain must tell one story about where the ground is.

B-135 · REQUIRED · terrain - Do not put a building in water: the terrain at every footprint point of every ground-contact level is at or above `extensions.terrain.seaLevelMetres`, and `FootprintInWater` is not true. Terrain below sea level is seabed.

B-136 · RECOMMENDED · terrain - Do not let the building float or sink: no ground-contact level's floor is above the terrain by more than `extensions.terrain.floatToleranceMetres` at every footprint point of that level, or below it by more than `extensions.terrain.buryToleranceMetres` at every one. A house on a slope may stand above the ground at one edge and below it at another; that is allowed. A building above or below the ground everywhere was placed at the wrong height. The check is an estimate from the footprint points, so it starts as a warning; a consumer that relies on it raises it in `ruleLevels`.

B-137 · REQUIRED · terrain - Pin the terrain source in a property set `Pset_Declaration_Terrain` on the terrain element: `Role` (`derived-context`), `PackageName`, `PackageVersion`, `PackageSha256`, `PatchSha256`, `PatchLevel`, `PixelSizeMetres`, `ErrorBoundMetres` (at most `extensions.terrain.maxErrorBoundMetres`), `Frame` and `VerticalDatum`. Heights near water can change between source versions, so a delivery says exactly which version it was built on and never carries numbers from an earlier one.

B-138 · RECOMMENDED · terrain - Keep the terrain within `extensions.terrain.maxTriangles`. Terrain triangles do not count against `budgets.recommendedTriangles`; they still count against the hard limits of B-05.

B-139 · RECOMMENDED · terrain - Keep water out of the file and declare it as metadata in `Pset_Declaration_Site` on the `IfcSite`: `SeaLevelMetres`, `FootprintInWater` and, when known, `DistanceToShoreMetres`. Sea and shoreline belong to the terrain source; the file only says how the building relates to them.

B-140 · RECOMMENDED · terrain - State the frame of the terrain numbers: `Pset_Declaration_Terrain.Frame` says that the file frame is x east, y north, z up and relative to the lowest finished floor. Never write coordinates in a consumer's runtime frame.

B-141 · RECOMMENDED · terrain - Declare every level that touches the ground with `GroundContact` true in `Pset_Declaration_Level`; without a declaration the lowest level is the only ground-contact level. On a stepped or terraced site several levels meet the ground, each at its own height, and the terrain source needs each footprint and floor elevation to grade them. The checker warns about an undeclared level that sits on the terrain.

Guide 2: Interior Decorator's Guide

For interior decorators, material authors and texture artists who define finishes, wall coverings, floor and ceiling treatments, accents and trims, and who deliver a reusable material library.

Rule levels and the profile mechanism are explained in the README; profile parameters appear in `backticks` (full list: 07-profile-parameters.md). Read sections 1 to 4 of guide 1 first: finishes attach to the rooms and element classes defined there.

1. How consumers decide what a surface looks like

Consumers differ here, which is why this guide is built around a portable method.

1. **IFC carries a colour and an opacity per surface style.** `IfcSurfaceStyle` with a rendering colour and a transparency is read by every common reader. Texture images, texture coordinates and physically based settings are read by some and ignored by many, so a file that relies on them looks different (or flat) in other tools.
2. **A finish schedule says what each style means.** You deliver a small JSON file that maps every (element class, colour, opacity) key used in the IFC to a finish: which texture set, how large one tile is, how strongly to tint it. A consumer that has its own materials can follow the schedule; one that does not still shows your colours.
3. **Texture sets are delivered separately,** as folders of images with a description file, so that any consumer can load them with its own mapping (projection, UVs, atlases).
4. **Floors, ceilings and walls use faces.** A finish can describe up to three face slots: `top` (faces pointing up, the walking surface), `side` (edges) and `bottom` (faces pointing down, which is the ceiling of the room below); most classes use `all`.
5. **Only emissive parts glow.** In consumers that turn fixtures into lights, the glow comes from the fixture part the profile names (guide 1, section 11), not from a colour.

What you control is: the class of each element, the colour and opacity you give it, the finish schedule, and the texture sets.

2. Colour rules

D-01 · REQUIRED - Give each distinct finish its own colour within an element class. The schedule keys finishes by (class, colour, opacity); two finishes with the same key are one finish. Colours that differ by less than 1/255 in every channel are the same colour.

D-02 · REQUIRED - Give the same finish the same colour and opacity on every element that uses it. A small drift (from re-rounding in another tool) creates a new key that has no schedule entry.

D-03 · REQUIRED - When you deliver a finish schedule, include an entry for every (class, colour, opacity) combination that appears in your IFC files. The checker lists them (`finishKeys`).

D-04 · RECOMMENDED - Use opacity below 1 only for glass and other see-through things (about 0.4 to 0.6 for glazing). Consumers treat anything with an opacity below 1 as see-through. A walkable glass floor is a slab with a glass colour and opacity.

D-05 · RECOMMENDED - Keep the palette small: at most `budgets.maxDistinctColours` combinations (default 50). Every combination is one more material to map, load and draw.

D-06 · RECOMMENDED - Treat the IFC colour as a design tint: the schedule says how strongly it is blended into the texture (0 means the texture alone, 1 the colour fully). Pick a colour near the intended look, not a code colour such as pure red.

D-07 · RECOMMENDED - Avoid pure black, pure white and very dark large surfaces: they clip or disappear under typical lighting. Keep large floors and walls between about 0.25 and 0.85 in mean linear luminance unless a dark look is the point.

D-08 · RECOMMENDED - Choose neutral tints for fabrics; strongly coloured tints shift under lighting.

3. Floors, ceilings, wall coverings, accents and trims

Floors

D-10 · RECOMMENDED - When you deliver a finish schedule, give every room that has a floor a floor finish: list them in the `floors` section by `Occupant`, by `RoomId` list or by pattern, plus a default. A room without a rule gets whatever the consumer defaults to.

D-11 · RECOMMENDED - Also write a text property `Finish` on each room (for example `light warm-grey resilient floor; pale ivory walls`), so that the intent stays with the room.

D-12 · RECOMMENDED - Change floor finishes at door lines, not mid-room, and keep a visible threshold between unlike finishes.

D-13 · RECOMMENDED - Choose low-gloss, light floors for rooms people work in: dark glossy floors read as black under many lighting set-ups.

Ceilings

D-20 · RECOMMENDED - Decide ceilings through the slab or roof above: the `bottom` face of the slab above is the ceiling of the room below. To get a different ceiling finish give that slab a different colour, and describe its `bottom` slot in the schedule. A ceiling can also be an `IfcCovering` (D-31).

D-21 · RECOMMENDED - Keep ceiling finishes matte and light; glossy ceilings amplify lamp hot spots.

Wall coverings

D-30 · RECOMMENDED - Express a wall finish through the wall element's own colour (one colour per finish and per class). It works in every consumer.

D-31 · RECOMMENDED - Use an `IfcCovering` (`PredefinedType` `CLADDING`, `FLOORING`, `CEILING` or `WRAPPING`) only when a finish covers part of a wall, or a surface that has no element of its own. Model it as a shell at least 10 mm thick whose back face sits exactly on the surface (touching, not overlapping), so no two faces share a plane and face the same way. Support for `IfcCovering` varies between consumers, so confirm it with the consumer first.

D-32 · RECOMMENDED - Do not rely on a texture pattern starting at an edge: consumers project textures differently. Design coverings that tolerate any start position, or model panels as geometry when joints must line up.

Accents and trims

D-33 · RECOMMENDED - Make skirtings, cornices, door frames, handrails and accent bands separate elements with their own colour, at least 1 cm thick. Elements thinner than about 5 mm flicker against what they sit on.

D-34 · RECOMMENDED - Accent features (a feature wall, a coloured door, a timber panel) are a colour on the element in the same class as its neighbours, not an image inside the IFC.

Glass

D-35 · RECOMMENDED - Describe glass by colour, opacity and three optional settings: base opacity, edge reflection (`fresnelAlpha`) and smudge roughness (`smudgeRough`). Keep glass light and clear: dark or green glass reads as black or milky in many viewers.

4. Texture sets

A texture set is a folder of images plus a description; it is the unit of a material library.

D-40 · REQUIRED - Deliver every texture set as a folder `<setName>/` containing the maps the profile lists (`textures.maps`, default `albedo.png`, `normal.png`, `orm.png`) and `set.json`.

D-41 · REQUIRED - Use these encodings, so that every consumer reads the maps the same way:

Map	Content	Encoding
`albedo.png`	base colour	8-bit RGB PNG, **sRGB** encoded, no alpha, no baked lighting or shadows
`normal.png`	surface relief	8-bit RGB PNG, **linear** data, tangent basis, flat surface = (128, 128, 255); green channel direction as the profile states (`textures.normalConvention`: `+Y` up, or `-Y`)
`orm.png`	packed material data	8-bit RGB PNG, **linear** data. **R** = ambient occlusion (255 = unoccluded), **G** = roughness (0 = mirror, 255 = fully rough), **B** = metalness

D-42 · RECOMMENDED - Make the texture tile seamlessly in both directions; consumers repeat it across large surfaces, and a seam repeats with it. Check the three-by-three preview.

D-43 · REQUIRED - Declare the physical size of one tile in `set.json` as `tileMetres`, so that a consumer can scale it correctly.

D-44 · RECOMMENDED - Use square, power-of-two images, all maps the same size, one of `textures.sizesPixels` (default 512, 1024, 2048). Larger maps cost memory in every view.

D-45 · RECOMMENDED - Choose the tile size so the texel density is inside `textures.densityPixelsPerMetre` (default 250 to 2500 pixels per metre): for 1024 pixels that is a tile of about 0.4 m to 4 m.

D-46 · RECOMMENDED - Avoid fine, regular weaves and stripes with a tile under 0.5 m: they produce moire at grazing angles.

D-47 · RECOMMENDED - Declare `tintMode` in `set.json` (one of `textures.tintModes`): `multiply` when the albedo is a near-neutral light map meant to be tinted by the IFC colour, or `authored` when the colour in the map is final.

D-48 · RECOMMENDED - Keep the pattern free of one-off features (a single stain, one knot) that would repeat visibly, and do not bake low-frequency variation in; consumers can add it.

D-49 · RECOMMENDED - If a pattern has a direction (planks, brushed metal, grain), say which image axis it follows in `set.json` (`direction`: `u`, `v` or `none`); consumers decide how directional patterns are oriented on floors and walls.

D-50 · RECOMMENDED - Check your tiles in a three-by-three arrangement and at a long viewing distance: fine relief that reads as speckle should be softened.

set.json

```
{
  "name": "plaster_fine",
  "version": "1.0.0",
  "sizePixels": 1024,
  "tileMetres": 2.0,
  "tintMode": "multiply",
  "direction": "none",
  "normalConvention": "+Y",
  "description": "fine trowelled plaster, low sheen",
  "maps": { "albedo": "albedo.png", "normal": "normal.png", "orm": "orm.png" },
  "generator": "optional: script and seed that reproduce the set"
}
```

5. The finish schedule

The finish schedule tells a consumer what each colour and each room floor means. The machine-readable shape is `schemas/finish-schedule.schema.json`; a starting file is `templates/finish-schedule.template.json`.

D-60 · REQUIRED - Deliver `finish-schedule.json` with these sections: `format` (`finish-schedule/1`), `delivery`, `version` (SemVer), `materials[]` (one entry per (class, colour, opacity) key: `key`, `role`, `usedBy`, `faces`), `floors[]` (ordered floor rules: `where` and `finish`), and `libraries[]` (the texture-set folders it names, with versions).

D-61 · REQUIRED - Name only texture sets that exist: each face entry names a set from your delivery or from a library the profile or consumer publishes. Settings are optional:

Setting	Meaning
`set`	texture-set name
`tint`	`"ifc"` (the element's colour), a colour such as `"#5a5654"`, or `"none"`
`tintStrength`	0 to 1: how far the tint replaces the texture's own colour
`tileMetres`	override the set's tile size
`normalScale`	strength of relief (typically 0.4 to 1.3)
`roughMul`, `metalMul`	multipliers on roughness and metalness
`macroStrength`, `macroScaleMetres`	optional low-frequency variation to hide repetition
`aoStrength`	strength of baked occlusion

Setting	Meaning
`sheen`, `sheenTint`	fabric edge response
`offsetMetres`	shift the pattern so neighbouring materials do not repeat in step

D-62 · RECOMMENDED - Mark materials that must stay as they are (emissive strips, planning markers) with `"keep": true` and a reason instead of giving them a texture.

D-63 · RECOMMENDED - Treat the schedule as a proposal: a consumer applies what it can. Do not depend on any single setting being honoured exactly.

A short example

```
{
  "format": "finish-schedule/1",
  "delivery": "example-room-1.0.0",
  "version": "1.0.0",
  "materials": [
    { "key": { "class": "IfcWall", "colour": "#d9d2c3", "opacity": 1.0 },
      "role": "room walls", "usedBy": "all rooms",
      "faces": { "all": { "set": "plaster_fine", "tint": "ifc", "tintStrength": 0.55, "normalScale":
    0.6 } } },
    { "key": { "class": "IfcSlab", "colour": "#c9c4b8", "opacity": 1.0 },
      "role": "floor slab", "usedBy": "all levels",
      "faces": { "top": { "set": "concrete_polished", "tint": "ifc", "tintStrength": 0.25 },
                 "side": { "set": "board_concrete" },
                 "bottom": { "set": "ceiling_gypsum" } } }
  ],
  "floors": [
    { "id": "floor-default", "where": { "occupant": "*" }, "finish": { "set": "oak_plank_wide" } }
  ],
  "libraries": [ { "name": "plaster_fine", "version": "1.0.0" } ]
}
```

6. Palettes

D-70 · RECOMMENDED - Build the palette from a few families and give each organisation or zone a distinct tint rather than a distinct pattern.

D-71 · RECOMMENDED - Describe the intent in words as well as colours ("light warm-grey resilient floor; pale ivory washable walls; low gloss"), so that a consumer with its own material library can choose.

D-72 · RECOMMENDED - Do not use texture or colour to carry meaning that labels already carry (room names, identifiers, directions). Use finish to make places feel different, not to label them.

7. Delivering a reusable material library

A library is a set of texture sets plus a finish schedule, with a manifest (guide 4). It is reusable when a different building can name your sets in its own schedule. `finishes/library.json` (`schemas/material-library.schema.json`) lists the sets and the schedule.

D-80 · REQUIRED - Record the source and licence of every texture set and every image in it in a `SOURCES.csv` next to the library with the columns `asset,type,source,licence,notes`. State whether the set is procedural (and which script and seed make it) or derived from photographs (and whose). Include no image whose licence you cannot state.

D-81 · REQUIRED - Include the SHA-256 hash of every file in the manifest (guide 4).

D-82 · RECOMMENDED - Prefer procedural sets with the generator and its seeds included; byte-identical output lets a consumer verify and regenerate the library and keeps the delivery small.

D-83 · RECOMMENDED - Name sets in lower case with underscores (`limewash_mottled`) and finishes with dotted names (`wall.limewash.warm`). Give each set its own SemVer; a changed look is a new version, never an overwrite.

D-84 · RECOMMENDED - Include a contact sheet (a PNG with each set's albedo, normal and roughness side by side, plus the tile size) so a reviewer sees the library at a glance.

D-85 · RECOMMENDED - Keep the library independent of any one building: no file name, label or set name should refer to a particular project.

8. Defects commonly seen

Defect	Cause	What to do
fabrics read pale or tinted	strongly coloured tint	neutral tint, lower sheen (D-08)
stone chips too bold	high chip contrast	soften relief and chip mask
woven wall cloth shows moire	small tile, strong relief	larger tile, lower relief (D-46)
carpets and floors read near black	very low albedo	lighter albedo (D-07)
parquet reads as staggered strips	pattern repeat scale	check the three-by-three view (D-50)
concrete pores read as spots	relief too sharp	soften relief (D-50)
glazing milky or black	dark or green glass colour	light clear glass (D-35)
blotchy walls	uneven lighting plus strong variation	moderate variation (D-48)

Exported appearance review

Review UV scale, grain direction, masonry corners, normal strength, surface-specific roughness/metallic response and clear-glass optics in the actual exported consumer artifact. An exporter or preview must not silently replace authored surfaces to make its image look correct. Inspect closeups and ordinary room views; use guide 9 to record evidence and limitations.

Guide 3: Furniture and Fixtures Guide

For designers who deliver furniture, equipment, fixtures and accents as reusable items that can be placed in any compatible consumer.

Rule levels and the profile mechanism are explained in the README; profile parameters appear in `backticks` (full list: 07-profile-parameters.md). Read sections 1 to 4 and 14 of guide 1 first, and section 1 of guide 2.

1. What consumers do with an item

An item reaches a consumer as an element inside a building's master IFC file. What happens next differs, but these things are common:

Stage	What typically happens	Consequence for you
Reading	the class of the element and its geometry are read; declaration properties are read by name	use the right class (section 2)
Material	class, colour and opacity select a finish (guide 2)	distinct finish, distinct colour
Collision	furniture is approximated by a simple shape, often **one axis-aligned box** around the element, while fixed equipment may collide as exact triangles	a chair can be a solid block; do not rely on walking under a table. The profile states the mode (`extensions.furnitureCollision`)
Standing points	simulators keep a start point clear of furniture by a margin	leave free floor (guide 1, B-33, with the walkability extension)
Interaction	most consumers have no sit points, drawers or moving parts for IFC items	item data beyond the model is for placement tools

An **item library** is a folder of single-item IFC files plus a description file per item. A building author places items into the master file as instances (section 8).

2. Classes and categories

F-01 · REQUIRED - Use these classes for items, because consumers and tools are written for them, and make the description's class agree with the model.

Item kind	IFC class	Typical consumer treatment
movable furniture (tables, chairs, benches, beds, shelves, sofas)	`IfcFurnishingElement`	simple shape or box; the default choice
fixed equipment, built-in units, counters, anything where the exact solid shape matters	`IfcBuildingElementProxy` (`PredefinedType` `USERDEFINED`, `ObjectType` set)	exact triangle mesh
light fixtures	`IfcElementAssembly` of two `IfcLightFixture` parts	see guide 1, section 11

Item kind	IFC class	Typical consumer treatment
signs and boards	`IfcElementAssembly`	see guide 1, section 12
rails and guards	`IfcRailing`	cheap geometry (guide 1, B-66)
lifts	`IfcTransportElement` (`ELEVATOR`)	usually static (guide 1, B-63)

Other classes are drawn but get no special treatment; ask before using them for items.

F-02 · RECOMMENDED - Set `ObjectType` to a category word from this list so that reports and schedules can group items: `table`, `desk`, `seat`, `bench`, `sofa`, `bed`, `shelf`, `cabinet`, `counter`, `partition`, `planter`, `equipment`, `appliance`, `lamp`, `sign`, `accent`, `other`.

F-03 · RECOMMENDED - Add a property set `Pset_Declaration_Item` with text properties `ItemId` (the library id, section 3), `ItemVersion` (SemVer), `Role` (what the item is for, for example `work table`), and optionally `Occupant` and `PlanningStatus`.

3. Item library format

One item is one folder.

```
library/
  library.manifest.json          the package manifest (guide 4)
  items/
    table.work.1600/
      table.work.1600.ifc        one IfcFurnishingElement at the origin
      table.work.1600.item.json  the description (section 3.1)
      preview.png                optional, 512 by 512
  finishes/                      optional: finish schedule and texture sets (guide 2)
```

F-10 · RECOMMENDED - Make each item file IFC4 (guide 1, B-01 to B-06) containing exactly one furniture or equipment element, placed at the origin with no rotation, in metres, with a stable `GlobalId`. The item file needs a project, the length unit and a geometric context, but no building, levels or rooms.

F-11 · RECOMMENDED - Give every item a library id of the form `category.name.size` in lower case (for example `seat.stool.450`), unique within the library and never reused for a different design. The folder, the files, `ItemId` and the description all use the same id.

F-12 · RECOMMENDED - Provide a description file `<ItemId>.item.json` for every item (section 3.1). The checker and reviewers use it; schema: `schemas/item.schema.json`.

3.1 The description file

```
{
  "format": "item/1",
  "itemId": "table.work.1600",
  "version": "1.0.0",
  "name": "Work table, 1.6 m",
  "category": "table",
  "class": "IfcFurnishingElement",
  "pivot": { "kind": "floor", "description": "centre of the footprint, on the floor" },
  "front": [1, 0, 0],
  "dimensions": { "x": 1.6, "y": 0.8, "z": 0.74 },
  "footprint": [[-0.8, -0.4], [0.8, -0.4], [0.8, 0.4], [-0.8, 0.4]],
  "collision": { "kind": "box", "min": [-0.8, -0.4, 0.0], "max": [0.8, 0.4, 0.74] },
  "clearance": { "front": 0.9, "back": 0.0, "left": 0.45, "right": 0.45, "top": 0.0 },
  "anchors": [
    { "id": "seat-1", "kind": "seat", "position": [0.0, -0.7, 0.45], "direction": [0, 1, 0] },
    { "id": "work-surface", "kind": "surface", "position": [0.0, 0.0, 0.74], "direction": [0, 0, 1]
    }
  ],
  "placement": {
    "mounting": "floor",
    "againstWall": false,
    "rotations": [0, 90, 180, 270],
    "roomUses": ["work", "meeting", "study"],
    "avoid": ["circulation"],
    "minWallDistance": 0.0
  },
  "variants": [
    { "id": "oak", "colour": "#ccbba0", "finish": "furniture.oak.veneer" }
  ],
  "budget": { "triangles": 12, "textures": 0 },
  "provenance": { "author": "organisation name", "licence": "stated licence", "source": "original" }
}
```

4. Dimensions, origin and pivot

F-20 · REQUIRED - Model in metres in the right-handed frame of guide 1 (B-02, B-10), with the item's own origin at its pivot (F-21).

F-20a · RECOMMENDED - Do not scale an item with an `IfcCartesianTransformationOperator` or any non-unit transform. Model it at its real size so that what you measure is what is drawn.

F-21 · RECOMMENDED - Put the pivot where the item meets the building, so placing it needs one point and one turn.

Mounting	Pivot (origin)	Local axes
`floor`	centre of the footprint, on the floor (lowest point at z = 0)	+X is the front (the side a user faces and the side that needs clear floor), +Z up
`wall`	centre of the back face, at the mounting height in `placement.mountHeight`	+X is the front, pointing away from the wall
`ceiling`	centre of the fixing plane (the top of the item)	+Z stays up; the item hangs below the origin, so its dimensions run to negative z
`surface`	centre of the contact area on the thing it stands on	+X is the front

F-22 · RECOMMENDED - Declare the bounding dimensions in the description and keep them equal (within 1 cm) to the measured size of the geometry.

F-23 · RECOMMENDED - Use true-to-life sizes: a work table is about 1.6 by 0.8 by 0.74 m, a chair about 0.5 by 0.55 by 0.85 m. Consumers do not scale items.

5. Placement points, anchors and where an item may be placed

Most consumers ignore this data; it lets a placement tool, or a different consumer, place items without a person.

F-30 · RECOMMENDED - Declare anchors in item coordinates (metres, relative to the pivot). Anchor `kind` values: `seat` (where a person sits; direction = the way they face), `surface` (a usable top), `reach` (where a person stands to use it), `storage`, `power`, `door-swing` (a volume to keep clear), `mount` (where it attaches). Give each anchor an `id` unique within the item.

F-31 · RECOMMENDED - Declare placement rules under `placement`: `mounting` (`floor`, `wall`, `ceiling`, `surface`), `againstWall`, `minWallDistance` (metres), `rotations` (allowed turns about the vertical axis, in degrees), `roomUses` (kinds of use it suits, free words matching the `Function` text of rooms), `avoid` (kinds of room it must not go in), and `mountHeight` for wall items.

F-32 · RECOMMENDED - Declare clearance minimums around the item: `front`, `back`, `left`, `right`, `top` (metres of free volume needed). As a guide: the profile's standing clearance on each side (`extensions.walkability.standingClearanceMetres`) and enough free floor in front of anything a person uses to reach it and pass. Keep aisles from seating to doors free.

F-33 · RECOMMENDED - Keep items off door openings and off the floor directly in front of a door; a table in the doorway fails the doorway check (guide 1, B-42).

F-34 · RECOMMENDED - Do not place items so that they cover a room's whole free floor (guide 1, B-33).

6. Collision and footprint

F-40 · REQUIRED · furnitureCollision - Assume the profile's collision mode (`extensions.furnitureCollision.mode`). With `box`, the consumer builds a single axis-aligned box around the whole element after placement: if that box fits badly (an L-shaped sofa, a corner desk) deliver several box-like elements instead of one. If exact shape matters (a sloped ramp, a stepped plinth) use an `IfcBuildingElementProxy`.

F-41 · RECOMMENDED · furnitureCollision - Place furniture with rotations in multiples of 90 degrees where you can; a box around a 45-degree table is larger than the table.

F-42 · RECOMMENDED - Declare the footprint polygon and the collision box in the description (section 3.1), so a checker and a human can see what a consumer will do.

F-43 · RECOMMENDED · furnitureCollision - Never make an item that must be walked under, through or between (an open pergola, a chair with gaps between its legs); in `box` mode it is a solid block.

F-44 · RECOMMENDED · walkability - Avoid low obstacles up to `extensions.walkability.stepHeightMetres` that are not obviously steps (a floor box, a cable cover): a walker steps over them unnoticed.

7. Polygon and texture budgets

F-50 · REQUIRED - Stay within the profile's limits for a whole delivery: `limits.maxFileMegabytes`, `limits.maxVertices`, `limits.maxTriangles`.

F-51 · RECOMMENDED - Keep each item small: at most `budgets.maxItemTriangles` triangles (default 2,000), far fewer for ordinary items. Do not model hidden undersides, fasteners or text.

F-52 · RECOMMENDED - Do not deliver textures with an item. Items take their look from the building's finish schedule (guide 2), keyed by class and colour. If a bespoke texture set is unavoidable, deliver it as a normal texture set (guide 2, section 4) and list it in the item description.

F-53 · RECOMMENDED - Use one or two surface colours on an ordinary item and up to four on a feature item; every distinct colour of every item adds a material.

8. Variants and instancing

F-60 · RECOMMENDED - Describe variants in the description file. A variant is a finish variant (a different colour; same item id, same geometry), a size variant (a different item id) or an option (different geometry, also a different id).

F-61 · RECOMMENDED - Place repeated items as instances: each instance is an element of its own (own `GlobalId`, own `ObjectPlacement`), sharing geometry through `IfcMappedItem` and `IfcRepresentationMap`. Rotate instances about the vertical axis only.

F-62 · REQUIRED - Give each instance its own unique `GlobalId` (guide 1, B-70) and a level to belong to (guide 1, B-13).

F-62a · RECOMMENDED - Place every instance inside the room it serves, and do not let it extend through a wall or a floor.

F-63 · RECOMMENDED - Record the item id and version on every instance (`ItemId`, `ItemVersion`, F-03) so that a revision of the library can be rolled out by replacing instances.

9. Fixtures

F-70 · REQUIRED · luminaires - Follow guide 1 exactly for light fixtures (section 11) and, when signboards are enabled, for signs (section 12). The declaration properties are read by name, and a missing required one makes the fixture unusable.

F-71 · RECOMMENDED · luminaires - Make a fixture's housing and emissive part separate parts so that only the emissive part glows. Keep it a simple disc or box, give the housing a dark neutral colour and the emissive part a warm off-white.

F-72 · RECOMMENDED - Other fixtures (hand rails, handles, hooks, bathroom fittings, planters) are small elements with their own colour, in the classes of section 2. Do not model plumbing or wiring.

10. What to avoid

- Rotated or L-shaped single elements that make a wasteful box (F-40, F-41).
- Items you can walk under, through or between (F-43).
- Low ledges and floor boxes that a walker steps over unseen (F-44).
- A table or shelf in front of a door or in an aisle (F-32, F-33).

- Per-item textures (F-52).
- Many near-identical colours across items (guide 2, D-01, D-05).
- Scaled instances (F-20a).
- Items that rely on moving parts, opening doors or drawers, or on sit points the consumer does not have.
- Hidden or tiny detail that costs triangles (F-51).

Guide 4: Delivery and Acceptance Guide

For everyone who sends or receives work: how to package it, what the manifest says, how versions and licences are recorded, how to check your own work before sending it, and how a receiving party accepts, returns or refuses it.

Rule levels and the profile mechanism are explained in the README. The scripts named here are in tools/; each prints what it checked and exits with status 1 when a REQUIRED rule fails. "The receiving party" is whoever takes the delivery: a consumer's operator, a client, an archive.

1. What you send

A delivery is one folder (and a zip of it) with these parts. Send only the parts your assignment needs.

```
example-room-1.0.0/
  manifest.json               what is in the package, with hashes (section 3)
  SHA256SUMS.txt              the same hashes in the plain checksum-file form
  ifc/
    example-room-master.ifc   the one coordinated master IFC file (buildings)
    alternates/               optional per-building files (not the master)
  tables/
    rooms.csv                 room table: ids, names, occupants, kinds (section 6)
    building.json             optional machine-readable building summary
  finishes/
    finish-schedule.json      what each colour and each floor means (guide 2)
    library.json              optional list of texture sets and the schedule
    SOURCES.csv               source and licence of every texture set
    textures/<set>/           albedo.png, normal.png, orm.png, set.json
  items/<itemId>/             <itemId>.ifc, <itemId>.item.json, optional preview.png (guide 3)
  terrain/                    only with the terrain extension: the height patch (patch.tif), its
    sidecar (patch.json),
                              and the site request (site-request.json) that produced it (guide 1,
    section 16)
  reports/                    the checker reports (section 5)
  notes/
    changes.md                what changed since the previous version
    open-points.md            anything you could not settle
```

X-01 · REQUIRED - Use this layout. Tools and reviewers find things by these folder names and roles. A building delivery has exactly one file under `ifc/` (not in `alternates/`).

X-02 · RECOMMENDED - Use lower-case ASCII file names without blanks, forward slashes in every path, and keep paths under 200 characters. Put nothing in the package that is not listed in the manifest.

2. Packaging the work

Build the package folder, write a short `package-meta.json` (start from `templates/package-meta.template.json`), and let the tool add the file list, hashes and sizes:

```
python3 tools/build_package.py init   example-room-1.0.0 package-meta.json
python3 tools/build_package.py verify example-room-1.0.0
```

`verify` is what a receiving party runs first, so run it yourself before sending.

3. The manifest

`manifest.json` has one entry for each of the following. Schema: `schemas/manifest.schema.json`.

Field	Content
`format`	always `delivery-manifest/1`
`delivery`	`id` (lower case, dots and hyphens), `title`, `version` (SemVer), `created` (UTC date and time in the form `2026-10-22T10:00:00Z`), `supersedes` (the version it replaces, or null), `kind` (a list of `building`, `materials`, `furniture`, `mixed`)
`author`	`organisation`, `role` (who answers queries), `contact` (a role mailbox or handle; personal data is not needed)
`profile`	`name`, `version` (and optionally `sha256`) of the consumer profile you built and checked against
`frame`	`units` (`metres`), `axes`, `levelOffsetMetres`, `localZero`
`ifc`	`schema`, `entityNumberStart`
`files[]`	for each file: `path`, `role`, `sha256`, `bytes`, optional `version`
`terrain`	only with the terrain extension: `package` (`name`, `version`, `sha256` of the terrain source version used), `patch` (`path`, `sha256`), `sidecar` (`path`)
`counts`	numbers the checker found (levels, rooms, triangles ...)
`provenance[]`	for each asset group: `asset`, `type`, `source`, `author`, `licence`, `notes`
`licence`	`statement` that covers everything not listed in `provenance`
`checks`	`tool`, `toolVersion`, `report` (path in the package), `result`

X-10 · REQUIRED - Put a `manifest.json` at the top of the package that matches the schema.

X-11 · REQUIRED - Record the SHA-256 hash (64 lower-case hexadecimal characters) and size in bytes of every file, and include a `SHA256SUMS.txt` in the plain two-column form (`hash`, two blanks, `path`). Hashes prove that what the receiving party holds is what you sent.

X-12 · REQUIRED - List every file in the package, and only those, in the manifest. (The manifest and the checksum file themselves are the exceptions.)

X-13 · RECOMMENDED - A building delivery has exactly one master IFC file (guide 1, B-03); other files are alternates.

X-14 · REQUIRED - Keep the whole package within the profile's cap, `package.maxPackageMegabytes`, when it sets one. Upload systems and mail gateways refuse larger packages.

X-20 · REQUIRED - Give every IFC file, texture set and library item a `provenance` entry (what it is, where it came from: `original`, `procedural` with the script and seed, or a named source; who made it; the licence that applies) when the profile requires provenance (`package.requireProvenance`, default true).

X-21 · REQUIRED - State a licence for every asset and a `licence.statement` for the package when the profile requires it (`package.requireLicence`, default true). A package whose licence terms are missing, or do not allow the receiving party to use, copy and adapt the work for the purpose of the assignment, cannot be accepted.

X-23 · REQUIRED · terrain - When the delivery has terrain, list the patch and its sidecar under `terrain/` with their roles, give each a provenance entry, and record the source package name, version and hash and the patch hash in the manifest `terrain` section, equal to what the sidecar states and to what the check report of the master file found in it, and keep the site request (its building origin, azimuth and floor height must be the file's). The terrain source can change between versions (shorelines in particular), so the delivery must say exactly which version its heights came from.

X-22 · RECOMMENDED - Deliver only content you are free to hand over. Do not include material that is restricted, reproduced from another party's work without permission, or that carries identifying marks of a third party. If part of your work is restricted, say so in `notes/open-points.md` and leave it out.

Machine-readable summaries

Two optional files make a delivery readable without opening the model or the schedule. Both have JSON Schemas, and `check_all.py` validates them when present (a problem is a RECOMMENDED finding, rule X-41).

File	Schema	Content
`tables/building.json`	`schemas/building-metadata.schema.json`	building id and name, frame, the master file, levels with elevations and room counts, occupants, room-id prefix
`finishes/library.json`	`schemas/material-library.schema.json`	the texture sets (name, version, path) and the finish schedule of a material delivery

4. Versions

X-30 · RECOMMENDED - Version the delivery, each texture set and each item with SemVer (`MAJOR.MINOR.PATCH`), and give every version a creation time. A version is the exact set of bytes in its manifest.

X-31 · RECOMMENDED - Choose the next version number by what changed.

Change	Example	Next version
corrections that change no identifier and no meaning	a wrong door height fixed, a colour nudged	PATCH (1.0.0 to 1.0.1)
added content with every existing identifier unchanged	a new room, a new item, a new finish	MINOR (1.0.1 to 1.1.0)
anything that renames, removes or re-means an identifier, or changes the frame	a room id changed, a level moved	MAJOR (1.1.0 to 2.0.0)

X-32 · RECOMMENDED - Write `notes/changes.md`: for each version, the ids added, changed and removed, in plain sentences.

X-50 · REQUIRED - Never change a delivered version in place. A correction is a new version with a new manifest and new hashes; two different files must never carry the same version number.

5. Acceptance checks

Acceptance is a list of checks. The first group is what you run yourself with the scripts; the second is what a receiving party typically checks, so you know what will be looked at.

5.1 Checks you run before sending

One command runs everything that applies to a deliverable and prints one JSON report (`schemas/check-report.schema.json`): `python3 tools/check_all.py PATH --json --profile PROFILE.json`, where PATH is a master file, a `finishes/` folder, an `items/` folder or a package folder. Without `--profile` the default profile is used, and the report says which profile it checked against. Every tool below also takes `--json` and `--profile`. Exit codes: 0 no REQUIRED failure, 1 at least one, 2 wrong command line, 3 unreadable input. The loop for fixing findings is in `REPAIR-LOOP.md`.

```
python3 tools/check_all.py ifc/MASTER.ifc --profile PROFILE.json --json --out reports/check_all.json
python3 tools/check_ifc.py ifc/MASTER.ifc --profile PROFILE.json --finish-schedule
    finishes/finish-schedule.json --report reports/check_ifc.json
python3 tools/check_texture_set.py finishes/textures/SETNAME --preview wrap.png --report
    reports/check_texture_set.json
python3 tools/check_item.py items/ITEMID --report reports/check_item-ITEMID.json
python3 tools/build_package.py verify PACKAGE_FOLDER
```

Script	What it checks	Rules
`check_all.py`	runs everything below that applies to the path	all
`check_ifc.py`	schema, units, text form, size and triangle limits, levels and containment, rooms (floor, identifiers, occupants, function), doorways, windows, floor slabs, stairs and risers, reservation volumes, zones, optional luminaire and signboard declarations, build volume, finish colours, identifiers	B-01 to B-120, D-03 to D-05
`check_texture_set.py`	maps present, 8-bit RGB PNG, same square size, tangent-basis normal, packed occlusion-roughness-metalness, tile size, texel density, luminance, wrap seam, `set.json` fields	D-40 to D-49
`check_item.py`	one item at the origin, class, declared dimensions against geometry, pivot, collision box, triangle budget, item id	F-01 to F-51
`build_package.py verify`	manifest against the schema, every hash and size, no unlisted files, one master, package size, provenance and licence, the check report belongs to the master and passed	X-10 to X-40

X-40 · REQUIRED - Run `check_ifc.py` on the exact master file you are sending, with the profile, and include its JSON report in the package. The report records the SHA-256 of the master file it checked and the profile it used; a report for a different file is not accepted. A delivery with any REQUIRED failure in the report is not sent.

Findings have a fixed shape, which a receiving party also uses when it sends work back (section 7):

```
{ "id": "B-41@1a2b3c4d", "rule": "B-41", "level": "REQUIRED", "status": "fail", "severity": "error",
  "location": { "file": "master.ifc", "element": "3Z2k6lcTnJFPCzw1CFlWEi" },
  "message": "IfcDoor Entrance door is not linked to an opening in a wall",
  "fix_hint": "Add an IfcOpeningElement in the wall with IfcRelVoidsElement ... and
    IfcRelFillsElement ..." }
```

`status` is `pass`, `info`, `warn` (a RECOMMENDED rule not met) or `fail` (a REQUIRED rule not met). `element` is the `GlobalId` of the product concerned.

Rules the script can only estimate say "estimate" in their message: the standing-point check (B-33), the sign text fit and legibility (B-101, B-102) and the riser check (B-61). A person confirms those.

5.2 Checks a receiving party typically makes

#	Check	What it looks at	Rule
1	package intake	every hash and size, the layout, the manifest	X-10 to X-14
2	reading	the master opens in an accepted schema edition with a metre length unit and within the limits	B-01, B-02, B-05
3	identity	`GlobalId` values and entity numbers are unique across everything held; room and zone identifiers are unique	B-22, B-70, B-71, B-81
4	rooms	each room has geometry and a floor; rooms can be classified	B-20 to B-26
5	doorways	each door's opening is found, the rooms either side are listed, the opening is clear	B-41, B-42
6	windows	window openings cut their walls	B-44
7	standing points	(walkability extension) every room has a free standing point	B-33, B-50
8	stairs and ramps	stairs connect levels; risers and slopes within the profile	B-60 to B-65
9	reservation volumes	provisions are marked, not solid	B-80
10	light fixtures	(luminaires extension) required properties, positions match	B-90 to B-93
11	signs	(signboards extension) boards fit their text and are legible	B-100 to B-102
12	materials	every colour has a finish; every named texture set exists with matching hashes	D-03, D-40
13	build volume	no vertex outside the volume or inside a keep-out	B-110
14	repeatability	the same input gives the same result; unchanged identifiers stay	B-07, B-75

6. Pre-submission checklist

Tick every line before sending. Lines marked R are REQUIRED rules; the others are RECOMMENDED.

Profile and frame

- ☐ R I know which profile I build for (or I used the default and said so) and the report names it.
- ☐ R The file is in an accepted schema edition, in metres, in the profile's frame (B-01, B-02, B-10).
- ☐ The file is in the placement the profile expects, and a `FrameNote` property says so (B-11, B-12).

Structure

- ☐ R Every level has a name and an elevation, and every element belongs to a level (B-13).
- ☐ R The file is ASCII only (B-04), within the limits (B-05), and every product yields triangles (B-06).
- ☐ R Every `GlobalId` is unique and stable (B-70). Entity numbers follow the profile's range when it assigns one (B-71).

Rooms

- ☐ R Every room is an `IfcSpace` with geometry and a flat floor (B-20, B-21).
- ☐ R `RoomId` values are unique and meet the profile (B-22, B-23); every room has one (B-26, required in profiles that say so).

- ☐ Every room has `Occupant` and `Function`, a full name, and areas and levels where known (B-24, B-25, B-27, B-28).
- ☐ Streets, paths and park areas are rooms too (B-29).
- ☐ The room table is complete (X-41).

Fabric

- ☐ R Doors and windows fill openings that void walls (B-41, B-44); doorways are clear (B-42).
- ☐ R Slab holes are openings (B-53); floors are slabs or sit on a floor the consumer supplies (B-50, B-51).
- ☐ R Every stair flight is an `IfcStairFlight` (B-60); every multi-level building has a stair (B-62).
- ☐ Risers and slopes are within the profile (B-61, B-64).
- ☐ Reservation volumes are marked (B-80, B-81).

Optional extensions (only when the profile enables them)

- ☐ R Luminaires: assemblies with housing and emissive part and all properties, in the consumer's global frame (B-90 to B-93).
- ☐ R Signboards: all properties, fitting text, honest reading distance (B-100 to B-102).
- ☐ R Walkability: free standing point in every room (B-33).

Finishes and items

- ☐ R Every (class, colour, opacity) combination is in the finish schedule (D-03).
- ☐ R Every texture set has its maps and `set.json` with `tileMetres` (D-40, D-41, D-43).
- ☐ I looked at the three-by-three wrap preview of every texture (D-42, D-50).
- ☐ Items are modelled to the guide: class, pivot, collision (F-01, F-21, F-40).

Package

- ☐ R The manifest, hashes, provenance and licences are complete; `build_package.py verify` passes (X-10 to X-21).
- ☐ R The check report is for the exact master file and the profile used, and shows no REQUIRED failure (X-40).
- ☐ The version number follows the table in section 4, and `notes/changes.md` is written (X-30 to X-32).
- ☐ `notes/open-points.md` lists anything I could not settle.

X-41 · RECOMMENDED - Deliver `tables/rooms.csv` with the columns `roomId`, `name`, `occupant`, `kind` (`room`, `circulation`, `observation`, `open`), `enclosed` (`yes` or `no`), `level`, `levelElevationMetres`, `floorAreaM2`, `clearHeightM`, `buildingId` and `layer`, and list every new `Occupant` string with the room-id prefix that goes with it and its kind, so a receiving party can add its rules before intake. Open areas are listed as rooms too. A starting file is in `templates/rooms.template.csv`.

7. How a receiving party accepts, returns or refuses

7.1 Outcomes

Outcome	When	What the sender receives
Accepted	every check in section 5 passes and review finds nothing to change	a confirmation naming the delivery id, version and the master file's hash
Accepted with notes	every REQUIRED rule passes, and the reviewer lists RECOMMENDED rules or defects worth fixing later	the same confirmation plus a findings list
Revision needed	one or more REQUIRED rules fail, or a defect makes the work unusable, but the delivery is otherwise sound	a findings list (section 7.3); the sender sends a new version
Rejected	the package cannot be used as it stands: a hash does not match, the file cannot be read, the schema or units are wrong, a hard limit is exceeded, the licence is missing or does not allow use, or the work was built to the wrong profile	the reason, and what to do before sending anything again

7.2 Steps

1. **Intake.** Package checks (hashes, layout, manifest, provenance, licence). A failure here is a rejection or an immediate return, because nothing else can be trusted.
2. **Reading and identity.** The master file is read; units, limits, identifiers.
3. **Rule data.** For any new `Occupant` string or room-id prefix in the room table, the receiving party adds its classification rules, and renames declaration properties to its own names with `tools/map_properties.py` and a mapping file it keeps itself (a profile's `propertyNames` and `propertyValues` are exactly that mapping). The sender's file stays neutral.
4. **Build and check.** The receiving party builds from the files and runs the checks of section 5.2.
5. **Review.** A person looks at finishes and wrap, signs, lighting, furniture clearances, and anything the scripts only estimate.
6. **Outcome.** One of the four outcomes above.

7.3 Findings and revision

The receiving party sends findings in the shape of section 5.1: the rule, its level, the product's `GlobalId`, and what was measured against what was expected. The sender answers each REQUIRED finding with a fix or a reason.

X-60 · RECOMMENDED - When you send a revision, change only what the findings and your own notes name, bump the version (section 4), list what changed in `notes/changes.md`, and keep every unchanged identifier. A revision that renames identifiers without being asked is a MAJOR change and is reviewed again from the start.

7.4 After acceptance

An accepted version is held by its hashes. If you later replace a building, deliver a new version of the master file that keeps the identifiers of everything that stays; references the receiving party holds (which room is a start room, which sign belongs to which building) then keep working. Nothing in an accepted version is edited by the receiving party: if it needs a change, it asks for a new version.

8. Submitting a delivery as a pull request

Outside contributors can offer a delivery (a building, interior, furniture, materials, fixtures or hardware) to this repository. The process, in one line: fork the repository, build the delivery in a branch of your fork under `deliveries/<slug>/`, and open a pull request against `main`. `08-submissions-and-ingest.md` walks through it; this section holds the rules the checks enforce. Limits are the `submission` group of the profile (`07-profile-parameters.md`); the numbers below are the defaults.

A submitted delivery is an ordinary package (section 1) with three differences: its folder is `deliveries/<slug>/` and the slug equals `delivery.id`; its manifest is named `delivery.json` and carries an extra `submission` object (`schemas/submission.schema.json`); and it may hold preview pictures under `previews/`. `python3 tools/check_submission.py deliveries/<slug>` checks the form, `python3 tools/check_all.py deliveries/<slug>` checks everything (it runs `check_submission.py` and the package checks).

X-70 · REQUIRED - Put a submitted delivery in `deliveries/<slug>/` with a `delivery.json` whose `delivery.id` equals the slug and whose `submission` object states the title, the submitter handle, the licence, the profile used, the tool versions and the attestation. Reviewers and the intake tool find a delivery by its folder and its manifest, and a handle ties it to its author.

X-71 · REQUIRED - Offer the delivery under a licence the profile lists (`submission.allowedLicences`; by default CC BY 4.0, or CC0 when the delivery says so). A delivery under other terms cannot be taken in, because anyone who uses a catalogue entry later needs the terms to be uniform and known.

X-72 · REQUIRED - Include only files of the types the profile lists (`submission.allowedExtensions`; by default ifc, glb, png, jpg, json, md, csv and txt), with no program, script or archive and no executable file mode. A delivery is data; anything that can run or unpack itself is a risk for every person and tool that opens the delivery.

X-73 · REQUIRED - Keep a delivery within the profile's limits: at most `submission.maxFiles` files (200), `submission.maxFileBytes` per file (25 MiB) and `submission.maxDeliveryBytes` in total (100 MiB). Review and intake run on shared machines with fixed time and memory.

X-74 · REQUIRED - Use only relative, portable paths: no leading slash, no `..`, no backslash or drive letter, names of letters, digits, dot, hyphen and underscore that do not start with a dot, no reserved device names, no two names that differ only by case, and no path longer than `submission.maxPathLength`, in the folder and in every path the manifest and the checksum list name. A path that leaves the delivery folder or collides on another system can overwrite or hide a file.

X-75 · REQUIRED - Include no symbolic link and no special file, only plain files and folders. A link can point outside the delivery and make a reader open a file that was never submitted.

X-76 · REQUIRED - Have the submitter attest in `submission.attestation` that the work is their own or they may offer it, that they may license it under the stated licence, and that it holds no restricted third-party content. The maintainers cannot check ownership, so the statement is on record with the delivery.

X-77 · REQUIRED - Make every file what its name says: PNG and JPEG images that are real images within `submission.maxImagePixels`, GLB files that are binary glTF 2 of the stated length, IFC files that are IFC text files, JSON that parses, text in UTF-8 without NUL bytes or active markup (script, iframe, object, embed). A renamed file can hide content from review and can crash a reader.

X-78 · RECOMMENDED - Add at least one preview picture under `previews/` (or `items/<id>/preview.png`) so a reviewer sees the work before opening a model.

8.1 Proofs: models and pictures that anyone can reproduce

A reviewer should not have to open a modelling program to see what a delivery is. Every submitted delivery therefore carries **proofs** in `proofs/`: binary glTF (GLB) exports of the whole model and of every component, a validation report for those exports, and a standard set of rendered pictures, all tied together by `proofs/manifest.json` (`schemas/proofs.schema.json`, format `proofs/1`). One command makes them: `python3 tools/build_proofs.py deliveries/<slug>` (export with `tools/export_glb.py`, validation with `tools/validate_glb.py`, pictures with `tools/render.py`); `python3 tools/check_proofs.py deliveries/<slug> --reproduce` checks them and rebuilds them to compare. Everything runs without a graphics card or a display, so the same files come out on a laptop and on a build server. The limits are the `proofs` group of the profile (`07-profile-parameters.md`); the numbers below are the defaults.

The GLB files use metres and Y up: a point of the IFC frame (Z up) becomes `(x, y, z) -> (x, z, -y)`. Colours become PBR metallic-roughness materials; a texture set becomes a swatch with its maps embedded. Zones and reservation volumes state a requirement, not a thing: when the profile sets `proofs.glbVolumes` to `exclude` (or `export_glb.py --volumes exclude` is used), they get no mesh and no node and are listed in the `extras.excluded` of the whole-model GLB, which `validate_glb.py --ifc` accepts as the written reason for a missing node (only those two kinds, never a wall or a slab). A consumer that moves or turns single things (a door about its hinge, a chair) sets `proofs.glbPivots` to `placement`: each product then has its own origin, the node carries the placement, identical shapes share one mesh, and `model.placements.json` lists every node; a profile may also ask for indexed meshes (`proofs.glbIndexed`). Two surfaces in one plane facing the same way fight for the same pixels (z-fighting); `validate_glb.py` warns when two nodes share more than `budgets.coincidentFaceMinAreaM2` of such area (an estimate, RECOMMENDED), `tools/coplanar.py` lists the pairs, and `proofs.glbCoplanar` set to `union` merges the overlaps inside one product. The pictures use fixed lighting and a fixed background and are always made from the delivered GLB files, never from anything else: a plan per level (cut at 1.2 m above the floor), four interior views for each room kind that is present, four exterior views, and for every component a contact sheet of turntable frames (a swatch view for a texture set).

X-79 · REQUIRED - Put a proof manifest at `proofs/manifest.json` (format `proofs/1`) that lists every GLB, the GLB validation report and every render with its SHA-256, size and the tool and version that made it, and put nothing else in `proofs/`. A reviewer, a catalogue and the intake tool need one file that says exactly which models and pictures belong to which source, and a way to see that none was changed afterwards.

X-80 · REQUIRED - Make every delivered GLB a valid binary glTF 2 file in metres with Y up, with no external reference, no unused buffer, image or material, finite and unit-length normals, tangents where a normal map is used, named scene-graph nodes, and the node, triangle and texture limits of the profile. A GLB that is malformed, in another unit or axis, or that points outside itself shows a consumer a different building than the IFC file does, or crashes it.

X-81 · REQUIRED - Deliver a GLB of the whole master model and one of every item, covering, fixture, hardware piece, door, window and texture set, each made from the delivered IFC files (their SHA-256 is recorded), together with a validation report that passes and names every GLB by its SHA-256. The GLB files are what the pictures are made from and what a consumer without an IFC reader opens, so they must correspond to the delivered source and be known to be valid.

X-82 · REQUIRED - Include renders that cover every level (a plan), every room kind present (the profile's number of interior views), the outside of the building, and every item, covering, fixture, hardware piece, door, window and texture set (a contact sheet). A picture set that leaves out a

level, a kind of room or a component lets a defect there go unseen.

X-83 · REQUIRED - Make every render from a GLB of the delivery and record in the manifest the SHA-256 of that GLB, of the picture itself, its pixel size, the camera and the lighting; each recorded GLB hash must be the hash of the delivered GLB file. A picture that cannot be traced to the exact model it shows proves nothing about it.

X-84 · REQUIRED - Keep every render within the profile's pixel and file-size limits (`proofs.minRenderWidth`, `proofs.maxRenderPixels`, `proofs.maxRenderBytes`) and the whole `proofs/` folder within `proofs.maxProofBytes`. Pictures are for looking at on shared machines; a very large one costs every reviewer, and a tiny one shows nothing.

X-85 · REQUIRED - Keep personal data and local paths out of images and GLB files: renders hold only the image chunks the profile allows, no image holds EXIF, XMP, a colour profile with text or text that names a person, a folder, a host or an address, and no GLB holds a path, an address, an e-mail address or a generator other than the set's. A file that records who made it, where, or on which machine leaks that to everyone who receives the delivery.

X-86 · REQUIRED - Make the proofs reproducible: rebuilding the GLB files from the delivered IFC files must give the same bytes, and re-rendering the delivered GLB files must give pictures that differ from the delivered ones by no more than `proofs.renderTolerance`; where the toolchain is not available the check says so and is skipped, not passed. Proofs that cannot be reproduced cannot be trusted, and a rebuild is the cheapest way to see that the pictures do show the model.

Acceptance of a submission is a merge to `main`; see `08-submissions-and-ingest.md` for what happens next. The checks cannot judge quality, ownership or taste: a person reviews those, and may decline a delivery that passes every check.

8.2 Consumer appearance and remote review

A reproducible proof is not automatically a useful visual review. Follow guide 9: inspect every authored space and connection, construction/detail closeups and actual exported materials in the intended consumer. Record renderer/light substitutions explicitly. Keep technical validity, rendered appearance and device behavior separate, and attach revision-bound findings and a resumable handoff.

Guide 5: Worked example

A complete, minimal delivery: one room with one door, one window, one table, one wall covering and one ceiling light, on one level, with a finish schedule, a texture set, a library item and a package. It is small enough to read in full, and it passes every check in guide 4 under both profiles that come with the set: `profiles/default.json` and `profiles/example-strict.json`.

The IFC file is produced by a script, not stored in this set, so that it always matches the script and the version of the IFC library you have installed, and so that every number below can be reproduced. To build and check everything in one go:

```
pip install -r tools/requirements.txt
python3 tools/run_example.py example-out
```

This writes `example-out/example-room-1.0.0/` (the package), runs every checker on it with the default profile, builds the manifest and ends with `build_package.py verify`. Run `python3 tools/selftest.py` to see the checker catch deliberate faults in the same file, and `python3 tools/agent_harness.py` to see a fresh agent build and pass a delivery from the set alone.

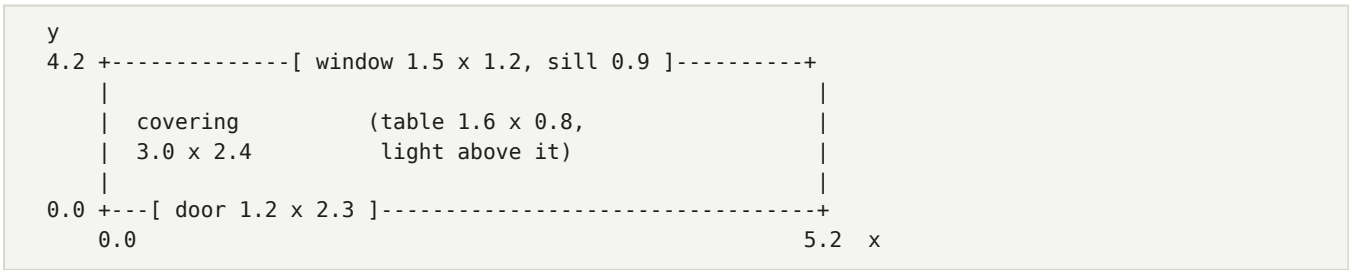
1. The profiles

Item	`default.json`	`example-strict.json`
frame offset	0	12.0 m (declared positions add it)
room identifier	up to 64 characters	at most 7 characters, required
entity numbers	any	from `#700001`
declaration property names	as in the guides	prefixed `Ex_` (the file is renamed by `tools/map_properties.py`)
extensions	all off	walkability, luminaires, signboards, furniture collision on
limits	250 MB, 5,000,000 triangles	100 MB, 3,000,000 triangles

To check the example against the strict profile, rename the properties and check with it:

```
python3 tools/map_properties.py master.ifc master-strict.ifc --mapping profiles/example-strict.json
python3 tools/check_ifc.py master-strict.ifc --profile profiles/example-strict.json
```

2. What is modelled



Element	Class	Size and position (file-local metres)
floor slab	`IfcSlab` `FLOOR`	5.2 by 4.2 by 0.3, top face at z = 0
roof slab	`IfcSlab` `ROOF`	5.2 by 4.2 by 0.3, underside at z = 3.0 (the ceiling)
south, north, west, east walls	`IfcWall`	0.2 thick, 3.0 high
door opening and door	`IfcOpeningElement`, `IfcDoor`	south wall, x from 0.8 to 2.0, 2.3 high
window opening and window	`IfcOpeningElement`, `IfcWindow`	north wall, x from 2.0 to 3.5, from 0.9 to 2.1 high, opacity 0.45
room	`IfcSpace`	inside faces: 4.8 by 3.8 by 3.0, floor at z = 0
table	`IfcFurnishingElement`, mapped item	1.6 by 0.8 by 0.74, pivot at the footprint centre, placed at (2.6, 2.1, 0)
wall covering	`IfcCovering` `CLADDING`	on the west wall's inside face: 10 mm thick, 3.0 m wide, from 0.1 to 2.5 high
luminaire	`IfcElementAssembly` with two `IfcLightFixture` parts	centred over the table, emissive part centre at (2.6, 2.1, 2.89)

3. Step by step

3.1 Frame, level and identity (B-10 to B-14, B-70, B-71)

The file is IFC4 in metres. The project carries a property set with `FrameNote` and `LevelOffsetMetres = 12.0`. One building and one level named `Example building - ground / +0 m` with elevation 0. Entity numbers start at `#700001` (the strict profile's range); every `GlobalId` is derived from a stable key, so the file is identical on every run.

3.2 Walls, openings, door and window (B-40 to B-44)

Each wall is an extruded rectangle. The door and the window each have an `IfcOpeningElement` that voids a wall (`IfcRelVoidsElement`) and is filled by the door or window (`IfcRelFillsElement`). The doorway is 2.3 m high and nothing crosses it. In the file:

```
#700198=IFCRELVOIDSELEMENT('1C_9SKyzTHcQf6qIs0XZ4T',$,$,$,#700125,#700197);
#700216=IFCDOOR('3Z2k6lcTnJFPCzw1CFlWEi',$,'Entrance
    door',$,$,#700213,#700215,$,2.3,1.2,.DOOR.,.SINGLE_SWING_LEFT.,$);
#700217=IFCRELFILLSELEMENT('29lMJRl7DVmRJwjxlT70xI',$,$,$,#700197,#700216);
```

3.3 Floor, roof and ceiling (B-50, B-53, B-54)

The floor is a slab whose top is the finished floor at z = 0, and the roof slab's underside is the ceiling. Many consumers draw no ceiling mesh, so that underside is what a person looks up at; guide 2 says how to give it a finish through the slab's `bottom` slot.

3.4 The room (B-20 to B-27)

```
#700273=IFCSPACE('1C1BaaS91KOgLuDEt1VXqX',$,'Example work room',$,$,#700270,#700272,'Example work
    room',.ELEMENT.,.INTERNAL.,0.);
#700274=IFCRELAGGREGATES('3jyuNv4XzQSRLalH7sDq_3',$,$,$,#700034,(#700273));
#700275=IFCPROPERTYSINGLEVALUE('RoomId',$,IFCTEXT('EX-001'),$);
#700276=IFCPROPERTYSINGLEVALUE('Occupant',$,IFCTEXT('Example Occupant'),$);
#700277=IFCPROPERTYSINGLEVALUE('Function',$,IFCTEXT('Work room'),$);
```

The room is an `IfcSpace` (the IFC name for a room or zone) aggregated under the level. Its geometry is an extrusion of the 4.8 by 3.8 floor outline up to 3.0 m. The properties are single values in a property set linked by `IfcRelDefinesByProperties`. `RoomId` is `EX-001`, six characters. The room table row repeats them, and the net area 18.24 m2 equals 4.8 times 3.8.

3.5 The table (guide 3)

The table's geometry is defined once, as a representation map with its origin at the footprint centre and its lowest point at z = 0, and the element is an instance of it (`IfcMappedItem`), placed at (2.6, 2.1, 0). In the item library the same table is a separate file, `items/table.work.1600/table.work.1600.ifc`, with a description file declaring its pivot, dimensions, collision box, side clearances, a seat anchor and placement rules.

3.6 The wall covering (D-30 to D-32)

The covering is a thin `IfcCovering` shell, 10 mm thick, whose back face sits exactly on the west wall's inside face (x = 0.2, the shell runs from x = 0.2 to 0.21). It has its own colour, `#8c6f4e`, so it becomes its own entry in the finish schedule. The simpler alternative for a whole wall is to give the wall element itself the finish colour. Support for `IfcCovering` varies, so confirm it with the consumer before relying on it.

3.7 The luminaire (optional extension; B-90 to B-93)

One assembly with a housing part and an emissive part whose name ends in `lens`. The assembly carries the declaration properties. The emissive part's centre is at local (2.6, 2.1, 2.89). Under a profile with a frame offset, the declared position is in the consumer's global frame, so the height gets the offset added once:

Property	Value
`LightPositionLocal`	`2.6,2.1,2.89` (optional, for checking)
`LightPosition`	`2.6,2.1,14.89` (2.89 + 12.0)
`LightAim`	`0,0,-1` (unit length)
`LightType`, `LightEnergy`, `LightRangeM`, `LightSpotDeg`	`spot`, 6, 8, 78
`LightColor`	`1,0.87,0.7` (warm)
`LightFadeBeginM`, `LightFadeLengthM`	40, 20
`LuminaireTarget`	`EX-001`

With the luminaires extension enabled, the checker confirms that the emissive part's centre plus the offset is within the profile's tolerance of `LightPosition`. Under the default profile (extension off) the fixture is simply a light fixture.

3.8 Colours and the finish schedule (guide 2)

Each (class, colour, opacity) combination needs a finish. The checker finds nine:

Class	Colour	Opacity	Used for
`IfcWall`	`#d9d2c3`	1.0	all four walls (one finish)
`IfcSlab`	`#c9c4b8`	1.0	floor slab
`IfcSlab`	`#e8e4da`	1.0	roof slab, whose underside is the ceiling
`IfcDoor`	`#977353`	1.0	door leaf
`IfcWindow`	`#91c8d0`	0.45	glass
`IfcFurnishingElement`	`#ccbba0`	1.0	the table
`IfcCovering`	`#8c6f4e`	1.0	the wall covering
`IfcLightFixture`	`#3d4348`	1.0	luminaire housing
`IfcLightFixture`	`#fff0cf`	1.0	luminaire emissive part

`finishes/finish-schedule.json` has one entry for each, and a floor rule for `Example Occupant`. The slab entries use all three face slots: `top` for the floor, `bottom` for the ceiling, `side` for the edge.

3.9 The texture set (D-40 to D-50)

`finishes/textures/plaster_example/` has `albedo.png`, `normal.png`, `orm.png` and `set.json` (1024 by 1024 pixels, 2 m per tile, so 512 pixels per metre; `tintMode` `multiply`). It is made by `tools/make_sample_texture_set.py` from seamless noise with a fixed seed. `check_texture_set.py --preview wrap.png` writes the three-by-three arrangement so you can see that the edges do not show.

3.10 Package, manifest and checks (guide 4)

`run_example.py` writes the check reports into `reports/`, writes `package-meta.json`, and runs `build_package.py init` then `verify`. The final package listing:

```
example-room-1.0.0/
  manifest.json   SHA256SUMS.txt
  ifc/example-room-master.ifc
  tables/{rooms.csv,building.json}
  finishes/{finish-schedule.json,library.json,SOURCES.csv}
  finishes/textures/plaster_example/{albedo.png,normal.png,orm.png,set.json}
  items/table.work.1600/{table.work.1600.ifc,table.work.1600.item.json}
  reports/{check_ifc.json,check_texture_set.json,check_item-table.work.1600.json}
  notes/changes.md
```

Expected output of the IFC check:

```
result: PASS (0 REQUIRED failure(s), 0 RECOMMENDED warning(s)); 1 levels, 1 rooms, 1 doors,
1 windows, 1 luminaires, 216 triangles, 136 vertices, 1 furniture, 9 finishKeys
```

4. Things to try

- Open `example-room-master.ifc` in any IFC viewer; it is also plain text.
- Edit `tools/make_sample.py`: rename `RoomId` to `EX-0001` and check with the strict profile (too long for 7 characters, B-23).
- Move the table to (1.4, 0.3) so that it stands in the doorway (B-42). The same fault is a case in `tools/selftest.py`.
- Remove the `IfcRelFillsElement` of the door (B-41), or change the emissive part's name to end in `glow` (B-90, strict profile).

- Change `LightPosition` height to 20 and see the mismatch (B-92, strict profile).

5. What the example leaves out

A free-standing sign (B-100), reservation volumes (B-80) and a library with several items. Each is a small extension of what is here; the rule text in guides 1 to 3 says how. Stairs, several levels, several rooms, open-air rooms, sloped sites and non-rectangular forms are in the larger examples below.

6. Larger examples

`python3 tools/make_fixtures.py out/` builds three more buildings with the authoring kit `tools/ifc_author.py`. Each passes `check_ifc.py` without warnings; `tools/test_fixtures.py` also runs them through the terrain tools and breaks each of the rules they exercise.

Fixture	What it shows
`tray-house`	two levels of cantilevered trays over a watercourse: stacked slabs, open-air terraces (B-143), a stair through a slab well sized for head-room (B-61, B-53), a stone core, parapets
`hillside-block`	a block stepped up a steep slope and turned 30 degrees from east: two ground-contact levels (B-141), footings and a retaining wall (B-142), a gable roof of sloped slabs (B-144), a stair that arrives on a roof terrace
`courtyard`	four wings round an open courtyard on one level: battered outer walls (B-144), flat roofs with a courtyard opening, parapets, open-air circulation reached by doors (B-47)

To see a building, run `python3 tools/preview.py MASTER.ifc OUT_DIR`: one plan per level and an isometric view.

How submissions and ingest work

This repository holds the guide set and it is also where outside contributors offer **deliveries**: buildings, interiors, furniture, materials, fixtures and hardware. A delivery is reviewed as a pull request. When the maintainers merge it, a separate tool turns the accepted delivery into separate, content-addressed component assets and a neutral catalogue. This page is for people and for software agents; the rules the automatic checks enforce are X-70 to X-86 in guide 4, section 8 (X-79 to X-86 are the proofs, below).

The path of a delivery

1. **Fork** this repository (your own copy on the hosting service).
2. **Build** the delivery in a branch of your fork, in the folder `deliveries/<slug>/`. The slug is lower-case letters, digits and single hyphens, for example `oak-study`. Follow guides 1 to 4 and the worked example (guide 5); `deliveries/example-room/` is a small accepted-looking delivery to copy from.
3. **Make the proofs** (`python3 tools/build_proofs.py deliveries/<slug>`, see "Proofs" below), refresh the manifest, then **check** it yourself: `python3 tools/check_all.py deliveries/<slug> --json` must exit 0 (this also runs `tools/check_submission.py` and `tools/check_proofs.py`). `tools/ci/run_checks.sh <slug>` runs exactly what the automatic check runs, including the rebuild of the proofs.
4. **Open a pull request** from your branch against `main`. Fill in the checklist in the pull request template.
5. **Automatic check.** A workflow checks the changed delivery folders (see below). Fix every REQUIRED finding and push again.
6. **Review.** A maintainer reads the delivery. The checks cannot judge quality, ownership or fit, so a delivery that passes can still be declined or sent back with notes.
7. **Acceptance is a merge to `main`.** Nothing else counts as acceptance, and no tool takes in a delivery from a branch or a fork.
8. **Ingest** runs afterwards, on the merged commit (see "Ingest"). The commit hash is the provenance of everything it produces.

Only the folder `deliveries/<slug>/` should change in a delivery pull request. One delivery per pull request. A correction after acceptance is a new pull request with a new version in the same folder (X-50: never edit a delivered version in place; add a new `delivery.version`).

What a delivery folder holds

A delivery is an ordinary package (guide 4, section 1) with these differences:

```
deliveries/oak-study/
  delivery.json            the manifest (schemas/manifest.schema.json) plus a `submission` object
    (schemas/submission.schema.json)
  SHA256SUMS.txt           hashes of every file, as in any package
  ifc/<name>-master.ifc    the master IFC file, when the delivery has a building or room
  items/<itemId>/          furniture, fixtures and hardware as items (guide 3): <itemId>.ifc and
    <itemId>.item.json
  finishes/                finish schedule, library.json and textures/<set>/ (guide 2)
  previews/                pictures of the work (png or jpg) so a reviewer sees it first
  proofs/                  GLB exports, their validation report, the standard renders and
    manifest.json (made by tools/build_proofs.py)
  reports/ notes/ tables/  the check reports, the change notes and the room table, as in guide 4
```

`delivery.json` is built with `python3 tools/build_package.py init DIR meta.json --manifest-name delivery.json`; the meta file carries the `delivery`, `author`, `profile`, `frame`, `provenance`, `licence` and `checks` parts of guide 4, section 3, and the `submission` object:

Field	Content
`title`, `summary`	what the delivery is, in a sentence
`submitter.handle`	the account the pull request is opened from (no personal data is needed)
`licence`	`CC-BY-4.0` (the default), or `CC0-1.0` when the delivery says so; the profile lists the licences it takes
`profile`	name and version of the profile the delivery was checked against
`toolVersions`	the tools and libraries used (this guide set's version, the IFC library)
`attestation`	`ownWork`, `mayLicense`, `noRestrictedContent`, all `true`

Contributed assets are offered under CC BY 4.0 unless the delivery states CC0. By opening the pull request the contributor attests that they own the work or may license it, and that it contains no restricted third-party content (X-22, X-76).

Proofs

Every submitted delivery carries **proofs**: models and pictures that anyone can look at without a modelling program, and rebuild to see that they are honest. `python3 tools/build_proofs.py deliveries/<slug>` makes all of them from the delivered files, in three steps, and writes `proofs/manifest.json` (`schemas/proofs.schema.json`, format `proofs/1`):

1. **Export** (`tools/export_glb.py`): `proofs/glb/model.glb` (the whole master IFC as a scene graph: `model` > `level:<name>` > `<kind>:<name>#<GlobalId>`), `proofs/glb/components/*.glb` (one per item and per furniture element, covering, fixture or luminaire assembly, hardware piece, door and window, exactly the GLB `ingest.py` stores for it) and `proofs/glb/materials/*.glb` (a 2 x 2 swatch of each texture set with its maps embedded). Units are metres and the axes are Y up: an IFC point (x, y, z) becomes (x, z, -y). Colours are PBR metallic-roughness materials. The files are deterministic and hold no path, time, user or host.
2. **Validate** (`tools/validate_glb.py`): structure (container, indices, accessors, images, materials, scene graph, nothing unused, no URI), conventions (units and axis note, generator, size, unit-length normals, tangents with a normal map, node names), the profile's budgets, and with `--ifc` each product's bounding box against the IFC file. The Khronos glTF Validator (Apache-2.0, pinned in `tools/khronos/package-lock.json`, `npm ci --prefix tools/khronos`) is run as well when it is installed. The report is `proofs/reports/glb-validation.json` (`check-report/1`, rule X-80).

3. **Render** (`tools/render.py`), from the GLB files only: a plan for each level (cut at 1.2 m), four interior views for each room kind present (kind = the room's `Function` property; the first two rooms of a kind by identifier), four exterior views, and a contact sheet of eight turntable frames for every component (four fixed frames for a texture swatch).

The renderer is a software rasteriser in numpy (about 400 lines): no graphics card, display, browser or system library, so the same GLB gives the same picture on a laptop and on a hosted runner. It draws flat-shaded triangles with a z-buffer, 2 x 2 supersampled, with base colours and the base colour texture; it does not draw normal maps, metal, roughness, shadows or reflections. The lighting, background, camera and sizes are fixed conventions, recorded for every picture (`tools/render.py` states them). A headless browser with a bundled viewer library, or Blender, would draw nicer pictures, but they depend on a graphics stack that differs between machines, so the same file would not give the same picture; the choice favours reproducibility over beauty. (Tried while choosing: headless Chromium with software WebGL runs here and repeats itself on one machine, but needs a browser, a vendored viewer library of some hundreds of kilobytes, and gives pixels that depend on the browser and graphics-layer build; Blender is optional high-quality software that is far larger than this repository and, with a graphics card, differs from machine to machine.) The manifest records, for every GLB, its SHA-256, size and source IFC hash, and for every picture its path, SHA-256, pixel size, camera, light, the hash of the GLB it was made from and the tool and version.

`python3 tools/check_proofs.py deliveries/<slug>` checks the manifest against the files (X-79), the GLB files against their sources and the validation report (X-81, X-80), the coverage of the pictures (X-82), the traceability of each picture to a delivered GLB (X-83), the size limits (X-84) and the metadata of every image and GLB (X-85); with `--reproduce` it also rebuilds the GLB files, compares the bytes, re-renders every picture from the delivered GLB and compares it within `proofs.renderTolerance` (X-86). Where the toolchain is not available it says so (an `info` finding) and skips; it never reports a skipped check as passed. Make the proofs again after any change to an IFC file, item or texture set.

Limits

Defaults, all parameters of the profile (`submission` and `proofs` groups, see `07-profile-parameters.md`): at most 600 files (proofs add a GLB and a sheet for every component; it was 200 before 2.3.0), 25 MiB per file, 100 MiB per delivery; file types ifc, glb, png, jpg, json, md, csv and txt only (csv and txt exist because the room table and `SHA256SUMS.txt` need them); no program, script, archive or executable file mode; no symbolic links; relative, portable, case-unique paths of at most 200 characters; images of at most 64 million pixels; renders of at least 256 and at most about 2 million pixels and 3 MiB, and `proofs/` of at most 64 MiB. A consumer that accepts submissions can set other values in its own profile.

The automatic check

`.github/workflows/check-delivery.yml` (a copy of the same steps without any workflow system is `tools/ci/run_checks.sh`) runs on pull requests that touch `deliveries/**`. It installs the pinned requirements (`tools/requirements-ci.txt`), then for each changed delivery runs `check_submission.py` and `check_all.py` under `profiles/default.json`, and again under `profiles/<name>.json` when `delivery.json` names a profile that exists there; once under the default profile it also runs `check_proofs.py --reproduce` (it rebuilds the proofs on the runner, which has no graphics card) and

`validate_glb.py` with the pinned Khronos glTF Validator (installed by `npm ci`, no secret needed). It uploads the JSON reports as an artifact, writes a summary to the run, and fails on any REQUIRED finding.

Pull requests from forks run with a read-only token and no secrets, and the workflow never uses `pull_request_target`. A first-time contributor's run may wait until a maintainer approves it. A fork's token cannot write comments, so the result is the run summary and the artifact; for pull requests from branches of this repository the workflow also posts the summary as a comment.

Ingest

`python3 tools/ingest.py deliveries/<slug> --commit <merged commit hash> --out <output folder>` is run by a maintainer or an automation, on a checkout of the merged commit of `main` (`--require-ancestor-of origin/main` makes it refuse anything else). It never uploads and uses no network. In order, it:

1. copies the delivery (plain files only) to a temporary folder, with a time limit (`submission.ingestMaxSeconds`) and an optional memory limit;
2. re-runs `check_submission.py` and `check_all.py` on the copy (the latter includes the stored-file checks of the proofs), and refuses on any REQUIRED finding;
3. splits the delivery into **components**: each item under `items/`; from the master IFC each furniture element, covering (wall, floor or ceiling), fixture or luminaire assembly, hardware piece, door and window; each texture set as a material set;
4. exports each geometry component as a binary glTF (GLB, units metres, Y up) and reads it back to verify it;
5. stores every file as `blobs/sha256/<hash>.<ext>`; identical bytes (for example a table in `items/` and the same table in the master, or the same part in two deliveries) are stored once;
6. compares the contributor's proofs with a fresh export from the delivered files (GLB hashes, render hashes, and that every component GLB of the ingest is among the proofs) and reports any mismatch in the result; the ingest result stays authoritative and nothing is refused for it. Each component links the contact sheet of its GLB (`proof-render` file), and the delivery record lists every render and the GLB files;
7. writes `catalogue/index.json` (one entry per distinct component, with every occurrence), `catalogue/deliveries/<slug>@<commit12>.json` and the handoff manifest `handoff/<slug>@<commit12>.json`, shaped by `schemas/catalogue.schema.json`.

Each component carries: kind and category; dimensions and bounds; the pivot (items keep the pivot their description declares, master elements are moved to the centre of their footprint at their lowest point); anchors; the material references that link surface colours to the finish schedule and to the material-set components; and, per occurrence, the delivery slug, commit hash, submitter, licence, source path and IFC identifier, and its placement in the building file. The output depends only on the delivery bytes and the commit hash, so a second run gives the same bytes. `--dry-run` shows what would be stored and what the output folder already holds, and writes nothing.

The handoff manifest lists the blobs to publish and the components with their files, licences and submitters. Publishing them anywhere is a separate step owned by whoever runs the catalogue; this repository defines no destination.

Limits of this version: the renders show base colours and one texture (no normal maps, metal or roughness); a texture set is shown as a swatch, not on a wall; components from the master IFC are exported as GLB only (no per-component IFC file; item components also keep their source IFC);

walls, slabs, stairs and roofs are not components, only furnishing, coverings, fixtures, hardware, doors and windows; rotations are recorded as an angle about the vertical axis; textures are carried as material sets and are not baked into the GLB.

For agents

Harness-neutral steps; each command exits non-zero on failure. Replace `UPSTREAM` with the repository you were asked to contribute to.

```
gh repo fork UPSTREAM --clone --default-branch-only       # your fork, cloned; or fork in the web
    page
cd REPO && git checkout -b delivery-SLUG
pip install -r tools/requirements.txt
# build deliveries/SLUG/ (skills/submit-delivery/SKILL.md), then:
python3 tools/build_proofs.py deliveries/SLUG                          # GLB files, validation
    report, renders, manifest; then refresh delivery.json
python3 tools/check_all.py deliveries/SLUG --json --out report.json   # exit 0 required
git add deliveries/SLUG && git commit -m "feat(deliveries): add SLUG" -m "What it is and what it was
    checked against."
git push -u origin delivery-SLUG
gh pr create --repo UPSTREAM --base main --title "Delivery: SLUG" --body-file
    .github/pull_request_template.md
```

Fill in the checklist honestly; do not tick the attestation boxes unless they are true, and do not set `attestation` fields to `true` for work you cannot vouch for. Never edit tools, profiles or schemas in a delivery pull request. Stop and report if the same REQUIRED finding remains after three runs (`REPAIR-LOOP.md`).

Environment review and asynchronous handoff

This procedure supplements guides 1–4. It adds review guidance, not new default rule levels. The consumer's profile supplies tolerances, clearance, camera, device and performance targets. A valid file can still depict a poor environment.

1. Agree the work and pin its inputs

Assign one owner per editable source or shared exporter. Separate model author, material author, consumer integration and independent review where practical. Work on branches; do not overwrite an accepted delivery or another worker's candidate. Independent rooms and assets can proceed in parallel. Serialize changes to shared generators, material contracts and publication manifests.

Record the guide/profile versions and hashes, source commit, editable sources, dependencies, exact build commands, units and coordinate transform. Distinguish the canonical model hash from an optimized consumer model hash. A screenshot needs the latter, its material/lightmap assets, renderer version and camera. Use `templates/environment-handoff.template.json` for a resumable work record. Reference the evidence manifest and frozen render-input manifest by path and hash; they bind captures to model, pose, lightmaps, renderer modules, cameras, method and diagnostic limits. Include those files in the portable handoff.

Deliver relative paths and immutable artifact addresses that collaborators can access. A working-machine path or symlink alone is not a handoff. Keep rights, font/texture dependencies, procedural seeds and storage requirements with sources. Never include credentials or depend on a contributor's running browser session.

2. Check construction and usable space before multiplying media

Inventory every authored room, exterior and connection. Review a complete set of directions and upward/downward joins, then relevant closeups. A few attractive hero views do not prove room coverage. List places not inspected and why.

Inspect openings, wall/roof/bay joins, thresholds, chimney bores and flashing, hardware attachment, furniture clearances, support contacts and actual object orientation. Check props against the real support surface, not an assumed floor height. Model masonry bonds and corner returns when joints must align.

Verify the requested open-door state against actual fixed surfaces and other open leaves. “Open” is not synonymous with “passable.” Do not hide a blocking leaf, enlarge a clearance result, or rotate a leaf into a wall to obtain a pass. Preserve the neutral rig and record hinge, parts and applied pose exactly once.

Use actual finished-floor geometry for steps, decks and ramps. Semantic level labels and large furniture bounding boxes can misrepresent walkable space. Evaluate the consumer's body clearance and all required connections; compute connectivity again after stair/threshold repairs. A

room viewpoint should lie on an accessible component, not a comfortable-looking isolated patch behind furniture. Separate photographic compositions from natural standing/entry viewpoints.

3. Review material authoring and exported appearance separately

Name the intended surface: paint, wood species, metal/coating, plaster, glass, fabric, stone, foliage. A solid painted finish is often correct; do not substitute a generic timber or noise texture merely because the exporter has one available.

Check UV orientation, real scale, repetition and seams at corners and close range. Inspect color, normal, roughness, metallic, emission and optical properties after export and import into the intended consumer. Retain material identity through batching. Texture-cache keys must include settings that change the resulting material.

Alpha coverage alone does not establish believable clear glass. Preserve the source's intended transmission, index of refraction and roughness using supported properties; keep mirror and painted/opaque surfaces distinct. Record unsupported features. Do not silently replace materials by name in a proof renderer: that can make a broken export appear correct. Label diagnostic fill lights and substitutions; they cannot prove the unmodified consumer's appearance.

4. Prove the lighting and renderer contract

Document whether diffuse direct/indirect illumination is baked or evaluated at runtime. Avoid counting both. Preserve source UVs separately from lightmap UVs; verify geometry identity, UV coverage, overlap, receivers and gutters after optimization.

When filtering Monte Carlo noise, keep contact/direct shadow detail out of an indirect-light smoothing operation. Filter independent atlas charts independently. Specify linear versus display-encoded values, range/scale, texture channels and tone mapping. Check pixel values after delivery encoding: lossy image coding can introduce colored blocks into unrelated atlas islands. An attractive source render does not prove an optimized map or a different renderer.

Review actual consumer views of dark rooms, bright windows, foliage backsides, glass, metals and contact shadows. A changed sky must agree with the baked light's direction and color convention. Record dark, noisy or implausible results as open findings; do not declare the pipeline finished because its scripts exited successfully.

5. Package evidence that another contributor can continue

For each finding record severity, affected objects/rooms, observation, artifact hashes, correction owner, attempted fix and the matching verification view. Keep baseline and candidate evidence separate. A changed artifact invalidates its old acceptance; record a superseding decision rather than editing away adverse evidence.

Change	Evidence to revisit
Geometry or pose	Collision/support, connectivity, room stops, UVs, bake, media and consumer integration
Materials or source UVs	Export/import parity, relevant bake contributions, optical closeups and media
Renderer, sky or lighting	Matched views, affected media and device measurements

Change	Evidence to revisit
Encoding/optimization	Decoded pixels/surfaces, attribute retention and consumer views
Labels or publication only	Affected links, accessibility, asset integrity and deployed behavior

Reuse a prior geometry/navigation result for a material-only revision only when an explicit equivalence check proves unchanged relevant geometry and poses. A matching vertex count is insufficient; changed triangle diagonals can matter.

Record reviewer independence and limits. Schema validation, image existence, reproducibility and visual acceptance are separate results. A diagnostic render, software renderer or emulated phone must not be represented as a physical-device test. Measure actual transfer, first usable frame, frame times and repeated-entry resource cleanup. Optimize from evidence without hiding quality loss or inventing a universal limit.

6. Resume and improve

The receiving contributor verifies artifact hashes, reads open findings and runs the recorded next command. Record the output revision, decisions and remaining work in the handoff; attach readable contact sheets and exact evidence files. Release owned processes and scratch storage without deleting editable masters. Do not claim an old job is running without checking its real handle.

When a defect escapes the process, add its general cause, a reproducible example and a source/export/review check to the appropriate guide or tool. Fix the shared recipe as well as the particular asset. Acceptance remains the receiving party's explicit, revision-bound decision under guide 4.

Repair loop

How to get a deliverable to zero REQUIRED findings. The same loop works for a building, a material library, a furniture library and a package. It needs only the tools in `tools/`; no network and no service.

The loop

1. **Run** one command for the deliverable, printing JSON:

   ```
   python3 tools/check_all.py PATH --json --out report.json
   ```

   (`PATH` is a master `.ifc` file, a folder with `finishes/`, a folder with `items/`, or a package folder with `manifest.json`. Add `--profile PROFILE.json` for a building so the frame, identifiers and volumes are checked.)

2. **Read** the report. Look at `result`, `exitCode` and `summary.requiredFailures` first. Then list the findings whose `status` is `fail`: each has `rule`, `message`, `location.element` (a `GlobalId`, when one product is concerned), `location.file` and `fix_hint`. For more on a rule, open its entry in `rules.json` (`statement`, `rationale`, `examples`, `related`).

3. **Fix** the source of the deliverable (the model, the schedule, the item, the package inputs), not the report. Fix the lowest-numbered structural findings first (`B-01` to `B-06`, `B-13`, `B-20` to `B-26`): later findings often disappear with them. Make one kind of change at a time.

4. **Rebuild** whatever the fix touched (re-export the IFC, rebuild the manifest with `build_package.py init`).

5. **Rerun** step 1 and compare. Repeat until `summary.requiredFailures` is 0 and the exit code is 0.

Then look at `warn` findings (RECOMMENDED rules). Fix the ones whose `fix_hint` is cheap; list the rest in `notes/open-points.md`.

Report fields

Field	Meaning
`result`	`pass`, `fail` or `error`
`exitCode`	`0` no REQUIRED rule failed; `1` at least one did; `2` wrong command line; `3` an input could not be read
`summary`	counts: `requiredFailures`, `recommendedWarnings`, `passes`, `info`
`findings[].id`	stable: the rule id, then a short hash of rule, file, element and message; the same problem keeps the same id between runs
`findings[].status`	`pass`, `info`, `warn` (RECOMMENDED not met) or `fail` (REQUIRED not met)
`findings[].fix_hint`	what to change, present on `warn` and `fail`
`reports[]`	the full report of each tool that `check_all.py` ran

To list only what must be fixed, read the file with any JSON tool, for example:

```
python3 -c "import json;r=json.load(open('report.json'));[print(f['rule'],f['location'],f['message']
    ,'->',f['fix_hint']) for f in r['findings'] if f['status']=='fail']"
```

Stop conditions

Stop and report instead of looping when any of these holds:

- The same finding `id` is still present after three runs in a row. Report the finding, what you changed, and why you think it did not help.
- The exit code is `3`, or `result` is `error`. Fix the path or the missing input; this is not a rule failure.
- A fix needs information that is not in the profile or the files (an `Occupant` string, a level offset, a licence statement, the assigned entity-number start). Ask the person who gave you the profile; do not invent it.
- More than ten runs without the number of REQUIRED failures going down.
- A finding seems wrong. Say which rule and element, and what you measured. Do not work around it.

Never do these to get a pass

- Edit a report, change a rule level, delete a check, or edit a tool to hide a finding.
- Change a value only to satisfy the checker when it makes the model less true (for example, widening a door in the file while the drawing says otherwise).
- Delete the element that is flagged without saying so in `notes/changes.md`.
- Run the checker on a different file than the one you deliver (rule `X-40` ties the report to the file by hash).

Worked mini-loop

Run 1 prints a failure:

```
{ "rule": "B-42", "status": "fail", "location": {"element": "3Z2k6lcTnJFPCzw1CFlWEi"},
  "message": "doorway Entrance door is blocked at 0.6 m by Work table, 1.6 m",
  "fix_hint": "Make the opening at least 1.9 m high and remove anything (sill, bar, furniture) that
    crosses the doorway centre line." }
```

Fix: move the table out of the doorway and re-export. Run 2 shows `summary.requiredFailures` 0 and exit code 0. Then rebuild the manifest (`python3 tools/build_package.py init PACKAGE_DIR package-meta.json`), run `python3 tools/check_all.py PACKAGE_DIR --json`, and deliver.

Rules index

Every rule of the four guides in one table. Generated by `tools/rules_index.py`; do not edit by hand.

The same rules, with machine fields, are in `rules.json`.

214 rules: 75 REQUIRED and 139 RECOMMENDED.

Guide	REQUIRED	RECOMMENDED	Total
1 Builder's and Architect's	33	69	102
2 Interior Decorator's	10	34	44
3 Furniture and Fixtures	6	27	33
4 Delivery and Acceptance	26	9	35
All	**75**	**139**	**214**

Rule	Level	Guide	Statement
B-01	REQUIRED	1	Deliver an IFC schema edition the profile accepts (`schema.allowed`, default IFC4).
B-02	REQUIRED	1	Declare the length unit as metres (an `IfcSIUnit` `LENGTHUNIT` `METRE`, or a prefixed or conversion-based unit that resolves to metres) and model in it.
B-02a	RECOMMENDED	1	Prefer plain metres (a `METRE` unit with no prefix) over a prefixed or conversion-based unit, which some tools mishandle.
B-03	RECOMMENDED	1	Deliver one coordinated master file per placement area, and send per-building files only as alternates.
B-04	REQUIRED	1	Keep the file plain ASCII: write every non-ASCII character with the format's escape (`\X2\00B7\X0\` for a middle dot, `\X2\2014\X0\` for a long dash).
B-05	REQUIRED	1	Stay within the profile's hard limits: `limits.maxFileMegabytes`, `limits.maxVertices` and `limits.maxTriangles`.
B-06	REQUIRED	1	Use geometry that every common reader can display: swept solids (`IfcExtrudedAreaSolid` with rectangle, circle, arbitrary closed or arbitrary-with-voids profiles), faceted breps or polygonal face sets, and mapped items.
B-07	RECOMMENDED	1	Make the export deterministic: the same inputs give a byte-identical file (fixed timestamps, stable ordering, `GlobalId` values derived from stable keys).
B-08	RECOMMENDED	1	Write each STEP entity on a single line when the profile asks (`file.singleLineEntities`).
B-09	RECOMMENDED	1	Keep the file below the profile's recommended size, `budgets.recommendedFileMegabytes`.
B-10	REQUIRED	1	Use a right-handed frame with Z up and the horizontal axes the profile states (`frame.horizontalAxes`), in metres.
B-11	RECOMMENDED	1	Author in the placement the profile expects (`frame.placement`): with `final`, local z = 0 is the finished floor of the lowest level and the consumer applies no offset or rotation, so author in the final placement frame.
B-12	RECOMMENDED	1	State your frame in the file: a property set on the `IfcProject` with a text property `FrameNote` ("metres; +X forward; +Y left; +Z up; local z = 0 at the finished floor of the lowest level") and a number `LevelOffsetMetres`.

Rule	Level	Guide	Statement
B-13	REQUIRED	1	Create one `IfcBuildingStorey` for every level of every building, with a `Name` and a numeric `Elevation` equal to the level's finished-floor height in your file's frame, and make every element belong to one: contained (`IfcRelContainedInSpatialStructure`), aggregated (`IfcRelAggregates`) or, for zones, referenced (`IfcRelReferencedInSpatialStructure`).
B-14	RECOMMENDED	1	Use the full hierarchy `IfcProject` > `IfcSite` > `IfcBuilding` > `IfcBuildingStorey`, one `IfcBuilding` per building, and level names that start with the building name (`Clinic - ground / +0 m`).
B-15	RECOMMENDED	1	Keep floor-to-floor heights regular so that stairs, lifts and openings line up between levels; make tall halls an exception you can name.
B-142	RECOMMENDED	1	Put nothing below the lowest finished floor except slabs, footings, piles, walls, stairs, ramps, openings, the terrain element and reservation volumes.
B-20	REQUIRED	1	Model every room as an `IfcSpace` (internal, `ELEMENT`) whose geometry is an upward extrusion of the room's floor outline: the lowest plane of the solid is the floor and the top is the underside of the ceiling or slab above.
B-21	REQUIRED	1	Give every `IfcSpace` geometry.
B-22	REQUIRED	1	Give every room that has a `RoomId` a value that is unique, and write it as an `IfcPropertySingleValue` inside an `IfcPropertySet` linked by `IfcRelDefinesByProperties`.
B-23	REQUIRED	1	Make room identifiers meet the profile: at most `identifiers.roomIdMaxLength` characters and matching `identifiers.roomIdPattern`.
B-24	RECOMMENDED	1	Give every room an `Occupant` (text): the organisation or use it belongs to.
B-25	RECOMMENDED	1	Give every room a `Function` (text) describing its use.
B-26	RECOMMENDED	1	Give every room a `RoomId`.
B-27	RECOMMENDED	1	Also supply, when known: `NetAreaSquareMetres` (number), `BaseLevelMetres` (number), `ClearHeightMetres` (number) and `Notes` (text).
B-28	RECOMMENDED	1	Give each room a complete, human-readable `Name`; names appear in lists and on labels.
B-29	RECOMMENDED	1	Declare streets, paths, plazas and park areas as rooms too, with an identifier and a `Function`, so that they can be referred to, lit and signed.
B-143	RECOMMENDED	1	Model an open-air room (terrace, courtyard, deck, path, plaza) as an `IfcSpace` with `PredefinedType` `EXTERNAL` and the boolean property `Enclosed` set to false, and an internal room with `Enclosed` true or absent.
B-30	RECOMMENDED	1	Let rooms on one level touch along wall centre lines but not overlap by more than the wall thickness; overlapping floors make "which room is this point in" ambiguous.
B-31	RECOMMENDED	1	Draw the room outline at the inside wall faces so the floor area is the usable area.
B-32	RECOMMENDED	1	Group rooms into buildings with a `BuildingId` property, and mark long-lived content separately from replaceable content with a `Layer` property (`permanent` or `interim`), so that a building can be swapped without touching its surroundings.
B-33	REQUIRED	1	Leave every enclosed room a floor area where a person can stand: at least `extensions.walkability.standingClearanceMetres` clear of furniture, equipment, rails and transport elements.
B-34	RECOMMENDED	1	Keep a free floor circle of `extensions.walkability.freeCircleMetres` in every room, and leave clear aisles from seating to doors.

Rule	Level	Guide	Statement
B-40	REQUIRED	1	Model walls as `IfcWall` (`IfcWallStandardCase` is also read), not as generic proxies; consumers select, colour and count by class.
B-40a	RECOMMENDED	1	Use one element per maximal straight run rather than one per room edge.
B-41	REQUIRED	1	Model every door as an `IfcDoor` that fills an `IfcOpeningElement` in the wall, linked with `IfcRelVoidsElement` (wall to opening) and `IfcRelFillsElement` (opening to door).
B-42	REQUIRED	1	Keep every doorway passable: its clear height is at least `geometry.doorMinClearHeightMetres`, and nothing solid (a sill, a bar, a glazed panel, furniture) crosses the opening centre line at the heights in `geometry.doorwayProbeHeightsMetres`.
B-43	RECOMMENDED	1	Make doorways at least `geometry.doorMinClearWidthMetres` wide.
B-44	REQUIRED	1	Model every window as an `IfcWindow` in an `IfcOpeningElement` linked with the voids and fills relations, so the wall is cut where the window is.
B-45	RECOMMENDED	1	Give glass a surface style with opacity below 1 (about 0.4 to 0.6), so that every consumer shows it as see-through.
B-46	RECOMMENDED	1	Give walls their real thickness; zero-thickness walls and double walls confuse area, collision and rendering.
B-47	RECOMMENDED	1	Make every room reachable through a doorway, an open edge or a stair from a circulation room.
B-48	RECOMMENDED	1	Do not put a door in a floor or roof; floor apertures are slab openings (B-53).
B-50	RECOMMENDED	1	Model every walkable floor, including walkable roofs, as an `IfcSlab` (`FLOOR` or `ROOF`) whose top face is the finished floor.
B-51	RECOMMENDED	1	Do not duplicate a floor the consumer already provides (`baseSurfaces`); two coplanar surfaces flicker.
B-51a	RECOMMENDED	1	Omit slabs that rest directly on a consumer-supplied floor, or make them very thin.
B-52	RECOMMENDED	1	Give slabs a realistic thickness; very thin slabs are fine only for surface layers such as paving.
B-53	REQUIRED	1	Cut every hole in a slab (stair wells, shafts, service drops) as an `IfcOpeningElement` linked by `IfcRelVoidsElement`, not as separate floor pieces that merely leave a gap.
B-54	RECOMMENDED	1	Model ceilings or roofs.
B-144	RECOMMENDED	1	Model a pitched or sloped roof with sloped `IfcSlab` `ROOF` pieces that follow the real pitch, and a wall whose faces lean (a battered wall) as one `IfcWall` with an inclined section, not as flat boxes or steps.
B-55	RECOMMENDED	1	Keep your own structure clear of openings in surfaces the consumer supplies (`baseSurfaces`), with the margin the profile states.
B-60	REQUIRED	1	Model each stair flight as an `IfcStairFlight` (a parent `IfcStair` without geometry is fine).
B-61	RECOMMENDED	1	Keep risers at or below `geometry.stairRiserLimitMetres`, with a landing and head-room; avoid winding and spiral stairs unless you have checked the consumer reads them.
B-62	RECOMMENDED	1	Give every multi-level building at least one real stair.
B-63	RECOMMENDED	1	Draw lifts as `IfcTransportElement` with `PredefinedType` `ELEVATOR`, with properties such as `Service`, `StopsMetres` and `ShaftWidthMetres`.

Rule	Level	Guide	Statement
B-64	RECOMMENDED	1	Keep every walked slope at or below `geometry.rampMaxSlope` (rise over run) except stairs, and give ramps a clear width at least as wide as a doorway.
B-65	RECOMMENDED	1	Do not leave low obstacles up to `extensions.walkability.stepHeightMetres` high (floor boxes, square-edged wedges) that a walker steps over unseen; ramp them or make them visible steps.
B-66	RECOMMENDED	1	Model rails (`IfcRailing`) with cheap geometry: simple posts and one rail section.
B-67	RECOMMENDED	1	Know which classes the consumer treats as not solid (`extensions.walkability.nonSolidClasses`) and model walk-through areas with those classes or with reservation volumes (B-80); everything else may block movement.
B-70	REQUIRED	1	Give every product a unique 22-character IFC `GlobalId` that does not change between exports.
B-71	RECOMMENDED	1	Number your IFC entities from the start the profile assigns (`identifiers.entityNumberStart`, when set) so that files from several authors can be merged without clashes.
B-72	RECOMMENDED	1	Write declaration properties as text, number or boolean values in `IfcPropertySingleValue` entries of an `IfcPropertySet` linked by `IfcRelDefinesByProperties` on the product itself.
B-73	RECOMMENDED	1	Use one descriptive property set per topic (`Pset_Declaration_Room`, ...) and put only declaration properties in it.
B-74	RECOMMENDED	1	If your organisation uses a classification scheme, attach it with `IfcRelAssociatesClassification`.
B-75	RECOMMENDED	1	Keep identifiers stable across revisions: an unchanged room, door or fixture keeps its `RoomId`, `GlobalId` and declaration ids.
B-76	RECOMMENDED	1	Give your own identifiers dotted prefixes (`arch.entrance`, `clinic.lobby`), at most 160 characters, no control characters.
B-80	RECOMMENDED	1	Declare a volume that must stay empty or must not be solid (a sight line, a clearance, a service corridor) twice: as an `IfcSpatialZone`, and (so it can be seen) as an `IfcBuildingElementProxy` with `PredefinedType` `PROVISIONFORSPACE`.
B-81	RECOMMENDED	1	Give every zone a unique identifier (`ZoneId` or, if missing, its `Name`) and its own geometry; a zone without geometry cannot be shown or selected.
B-81a	RECOMMENDED	1	Always supply `ZoneId`, plus `ZoneClass` (for example `observation-sightline`) and `Requirement` (a sentence saying what must stay clear).
B-82	RECOMMENDED	1	Use zones to record viewing intent (a downward sight line from a mezzanine); ordinary glass cannot be one-way, and who may look is a consumer decision, not a file property.
B-90	REQUIRED	1	Model each fixture as an `IfcElementAssembly` that aggregates two `IfcLightFixture` parts: a housing, and an emissive part whose `Name` ends in the profile's suffix (`extensions.luminaires.lensNameSuffix`).
B-91	REQUIRED	1	Put every property in `extensions.luminaires.requiredProperties` on the assembly, make `LuminaireId` unique, and use a `LightType` from `extensions.luminaires.lightTypes`.
B-92	REQUIRED	1	Write `LightPosition` in the consumer's global frame: the file-local point plus `frame.levelOffsetMetres` (same x and y, z plus the offset); the geometry stays file-local.

Rule	Level	Guide	Statement
B-92a	RECOMMENDED	1	Give the file-local position as well (`LightPositionLocal`) so a reviewer can check the offset was applied once.
B-93	REQUIRED	1	Make `LightAim` a unit vector (length 1 within 1e-6) in the consumer's frame.
B-94	RECOMMENDED	1	Give every enclosed room at least one fixture (set `LuminaireTarget`), between the floor and the ceiling, in warm colours, not pointed at glazing.
B-95	RECOMMENDED	1	Count the lights: every declared fixture may become a real light, and the consumer may cap them per room (`extensions.luminaires.maxPerRoom`).
B-100	REQUIRED	1	Model a board as an `IfcElementAssembly` carrying every property in `extensions.signboards.requiredProperties`, with a `SignboardKind` from `extensions.signboards.kinds`.
B-101	REQUIRED	1	Make the main text fit the board: allow about 0.55 times the capital height of width per character plus a margin of 12 % of the board height on each side; text that cannot be fitted at a capital height of 0.05 m is an error.
B-102	RECOMMENDED	1	Declare a reading distance your letters support: capital height in metres must be at least `pixels * readM / extensions.signboards.displayPixelsPerRadian`, with pixels from `extensions.signboards.minCapitalPixels` (primary text), and the distance within `extensions.signboards.readingDistanceMetres`.
B-103	RECOMMENDED	1	Declare an entrance board on the wall's outer face, not the wall centre plane, and keep lamps and trees away from its face.
B-110	REQUIRED	1	Keep all geometry inside the profile's build volume (`buildVolume`) and outside its keep-out regions (`keepOut`), when it gives them; check every vertex, not only footprints.
B-111	RECOMMENDED	1	Make solids closed and consistently oriented: no missing faces, no inside-out solids, no zero-area or duplicate faces.
B-112	RECOMMENDED	1	Avoid coplanar overlapping faces (two walls sharing a face, a covering lying exactly on a wall, a slab exactly on a floor).
B-113	RECOMMENDED	1	Use mapped items (`IfcMappedItem` with `IfcRepresentationMap`) for repeated parts.
B-114	RECOMMENDED	1	Do not model what cannot be seen or used: hidden interiors, bolts, hinge detail, text as geometry, curves with more than `geometry.maxCurveSegmentsPerCircle` segments per full circle.
B-115	RECOMMENDED	1	Do not model terrain, soil or relief in IFC unless the profile says how.
B-116	RECOMMENDED	1	Do not model a thin layer at a height where the consumer adds its own surface layer (floor finishes laid on the room floor); two layers at one height flicker.
B-120	RECOMMENDED	1	Stay under the profile's recommended triangle count, `budgets.recommendedTriangles`.
B-121	RECOMMENDED	1	Spend triangles where the viewer looks: entrances, interiors a person stands in, anything within arm's reach.
B-130	REQUIRED	1	Deliver the terrain as exactly one `IfcGeographicElement` with `PredefinedType` `TERRAIN`, contained in the `IfcSite`, whose body is one `IfcTriangulatedFaceSet`.
B-131	REQUIRED	1	Make the terrain a valid heightfield over a rectangular patch: every triangle faces up and has area, no edge belongs to more than two triangles, and the triangles cover the patch rectangle exactly once.

Rule	Level	Guide	Statement
B-132	REQUIRED	1	Make the terrain cover the building: the plan points of the slabs and walls of every ground-contact level, grown by `extensions.terrain.footprintMarginMetres`, lie inside the patch.
B-133	REQUIRED	1	Georeference the file with an `IfcMapConversion` from the model context to an `IfcProjectedCRS` named `extensions.terrain.crsName`, in metres, with the local X axis set to the building azimuth (a unit vector, east when the building is not turned), scale 1 and `OrthogonalHeight` equal to the height above sea level of the lowest finished floor (local z = 0).
B-134	REQUIRED	1	Set `IfcSite.RefElevation` to the terrain height at the building origin above sea level, within `extensions.terrain.siteElevationToleranceMetres` of the terrain element at that point.
B-135	REQUIRED	1	Do not put a building in water: the terrain at every footprint point of every ground-contact level is at or above `extensions.terrain.seaLevelMetres`, and `FootprintInWater` is not true.
B-136	RECOMMENDED	1	Do not let the building float or sink: no ground-contact level's floor is above the terrain by more than `extensions.terrain.floatToleranceMetres` at every footprint point of that level, or below it by more than `extensions.terrain.buryToleranceMetres` at every one.
B-137	REQUIRED	1	Pin the terrain source in a property set `Pset_Declaration_Terrain` on the terrain element: `Role` (`derived-context`), `PackageName`, `PackageVersion`, `PackageSha256`, `PatchSha256`, `PatchLevel`, `PixelSizeMetres`, `ErrorBoundMetres` (at most `extensions.terrain.maxErrorBoundMetres`), `Frame` and `VerticalDatum`.
B-138	RECOMMENDED	1	Keep the terrain within `extensions.terrain.maxTriangles`.
B-139	RECOMMENDED	1	Keep water out of the file and declare it as metadata in `Pset_Declaration_Site` on the `IfcSite`: `SeaLevelMetres`, `FootprintInWater` and, when known, `DistanceToShoreMetres`.
B-140	RECOMMENDED	1	State the frame of the terrain numbers: `Pset_Declaration_Terrain.Frame` says that the file frame is x east, y north, z up and relative to the lowest finished floor.
B-141	RECOMMENDED	1	Declare every level that touches the ground with `GroundContact` true in `Pset_Declaration_Level`; without a declaration the lowest level is the only ground-contact level.
D-01	REQUIRED	2	Give each distinct finish its own colour within an element class.
D-02	REQUIRED	2	Give the same finish the same colour and opacity on every element that uses it.
D-03	REQUIRED	2	When you deliver a finish schedule, include an entry for every (class, colour, opacity) combination that appears in your IFC files.
D-04	RECOMMENDED	2	Use opacity below 1 only for glass and other see-through things (about 0.4 to 0.6 for glazing).
D-05	RECOMMENDED	2	Keep the palette small: at most `budgets.maxDistinctColours` combinations (default 50).
D-06	RECOMMENDED	2	Treat the IFC colour as a design tint: the schedule says how strongly it is blended into the texture (0 means the texture alone, 1 the colour fully).
D-07	RECOMMENDED	2	Avoid pure black, pure white and very dark large surfaces: they clip or disappear under typical lighting.
D-08	RECOMMENDED	2	Choose neutral tints for fabrics; strongly coloured tints shift under lighting.

Rule	Level	Guide	Statement
D-10	RECOMMENDED	2	When you deliver a finish schedule, give every room that has a floor a floor finish: list them in the `floors` section by `Occupant`, by `RoomId` list or by pattern, plus a default.
D-11	RECOMMENDED	2	Also write a text property `Finish` on each room (for example `light warm-grey resilient floor; pale ivory walls`), so that the intent stays with the room.
D-12	RECOMMENDED	2	Change floor finishes at door lines, not mid-room, and keep a visible threshold between unlike finishes.
D-13	RECOMMENDED	2	Choose low-gloss, light floors for rooms people work in: dark glossy floors read as black under many lighting set-ups.
D-20	RECOMMENDED	2	Decide ceilings through the slab or roof above: the `bottom` face of the slab above is the ceiling of the room below.
D-21	RECOMMENDED	2	Keep ceiling finishes matte and light; glossy ceilings amplify lamp hot spots.
D-30	RECOMMENDED	2	Express a wall finish through the wall element's own colour (one colour per finish and per class).
D-31	RECOMMENDED	2	Use an `IfcCovering` (`PredefinedType` `CLADDING`, `FLOORING`, `CEILING` or `WRAPPING`) only when a finish covers part of a wall, or a surface that has no element of its own.
D-32	RECOMMENDED	2	Do not rely on a texture pattern starting at an edge: consumers project textures differently.
D-33	RECOMMENDED	2	Make skirtings, cornices, door frames, handrails and accent bands separate elements with their own colour, at least 1 cm thick.
D-34	RECOMMENDED	2	Accent features (a feature wall, a coloured door, a timber panel) are a colour on the element in the same class as its neighbours, not an image inside the IFC.
D-35	RECOMMENDED	2	Describe glass by colour, opacity and three optional settings: base opacity, edge reflection (`fresnelAlpha`) and smudge roughness (`smudgeRough`).
D-40	REQUIRED	2	Deliver every texture set as a folder `<setName>/` containing the maps the profile lists (`textures.maps`, default `albedo.png`, `normal.png`, `orm.png`) and `set.json`.
D-41	REQUIRED	2	Use these encodings, so that every consumer reads the maps the same way:
D-42	RECOMMENDED	2	Make the texture tile seamlessly in both directions; consumers repeat it across large surfaces, and a seam repeats with it.
D-43	REQUIRED	2	Declare the physical size of one tile in `set.json` as `tileMetres`, so that a consumer can scale it correctly.
D-44	RECOMMENDED	2	Use square, power-of-two images, all maps the same size, one of `textures.sizesPixels` (default 512, 1024, 2048).
D-45	RECOMMENDED	2	Choose the tile size so the texel density is inside `textures.densityPixelsPerMetre` (default 250 to 2500 pixels per metre): for 1024 pixels that is a tile of about 0.4 m to 4 m.
D-46	RECOMMENDED	2	Avoid fine, regular weaves and stripes with a tile under 0.5 m: they produce moire at grazing angles.
D-47	RECOMMENDED	2	Declare `tintMode` in `set.json` (one of `textures.tintModes`): `multiply` when the albedo is a near-neutral light map meant to be tinted by the IFC colour, or `authored` when the colour in the map is final.

Rule	Level	Guide	Statement
D-48	RECOMMENDED	2	Keep the pattern free of one-off features (a single stain, one knot) that would repeat visibly, and do not bake low-frequency variation in; consumers can add it.
D-49	RECOMMENDED	2	If a pattern has a direction (planks, brushed metal, grain), say which image axis it follows in `set.json` (`direction`: `u`, `v` or `none`); consumers decide how directional patterns are oriented on floors and walls.
D-50	RECOMMENDED	2	Check your tiles in a three-by-three arrangement and at a long viewing distance: fine relief that reads as speckle should be softened.
D-60	REQUIRED	2	Deliver `finish-schedule.json` with these sections: `format` (`finish-schedule/1`), `delivery`, `version` (SemVer), `materials[]` (one entry per (class, colour, opacity) key: `key`, `role`, `usedBy`, `faces`), `floors[]` (ordered floor rules: `where` and `finish`), and `libraries[]` (the texture-set folders it names, with versions).
D-61	REQUIRED	2	Name only texture sets that exist: each face entry names a set from your delivery or from a library the profile or consumer publishes.
D-62	RECOMMENDED	2	Mark materials that must stay as they are (emissive strips, planning markers) with `"keep": true` and a reason instead of giving them a texture.
D-63	RECOMMENDED	2	Treat the schedule as a proposal: a consumer applies what it can.
D-70	RECOMMENDED	2	Build the palette from a few families and give each organisation or zone a distinct tint rather than a distinct pattern.
D-71	RECOMMENDED	2	Describe the intent in words as well as colours ("light warm-grey resilient floor; pale ivory washable walls; low gloss"), so that a consumer with its own material library can choose.
D-72	RECOMMENDED	2	Do not use texture or colour to carry meaning that labels already carry (room names, identifiers, directions).
D-80	REQUIRED	2	Record the source and licence of every texture set and every image in it in a `SOURCES.csv` next to the library with the columns `asset,type,source,licence,notes`.
D-81	REQUIRED	2	Include the SHA-256 hash of every file in the manifest (guide 4).
D-82	RECOMMENDED	2	Prefer procedural sets with the generator and its seeds included; byte-identical output lets a consumer verify and regenerate the library and keeps the delivery small.
D-83	RECOMMENDED	2	Name sets in lower case with underscores (`limewash_mottled`) and finishes with dotted names (`wall.limewash.warm`).
D-84	RECOMMENDED	2	Include a contact sheet (a PNG with each set's albedo, normal and roughness side by side, plus the tile size) so a reviewer sees the library at a glance.
D-85	RECOMMENDED	2	Keep the library independent of any one building: no file name, label or set name should refer to a particular project.
F-01	REQUIRED	3	Use these classes for items, because consumers and tools are written for them, and make the description's class agree with the model.
F-02	RECOMMENDED	3	Set `ObjectType` to a category word from this list so that reports and schedules can group items: `table`, `desk`, `seat`, `bench`, `sofa`, `bed`, `shelf`, `cabinet`, `counter`, `partition`, `planter`, `equipment`, `appliance`, `lamp`, `sign`, `accent`, `other`.
F-03	RECOMMENDED	3	Add a property set `Pset_Declaration_Item` with text properties `ItemId` (the library id, section 3), `ItemVersion` (SemVer), `Role` (what the item is for, for example `work table`), and optionally `Occupant` and `PlanningStatus`.

Rule	Level	Guide	Statement
F-10	RECOMMENDED	3	Make each item file IFC4 (guide 1, B-01 to B-06) containing exactly one furniture or equipment element, placed at the origin with no rotation, in metres, with a stable `GlobalId`.
F-11	RECOMMENDED	3	Give every item a library id of the form `category.name.size` in lower case (for example `seat.stool.450`), unique within the library and never reused for a different design.
F-12	RECOMMENDED	3	Provide a description file `<ItemId>.item.json` for every item (section 3.1).
F-20	REQUIRED	3	Model in metres in the right-handed frame of guide 1 (B-02, B-10), with the item's own origin at its pivot (F-21).
F-20a	RECOMMENDED	3	Do not scale an item with an `IfcCartesianTransformationOperator` or any non-unit transform.
F-21	RECOMMENDED	3	Put the pivot where the item meets the building, so placing it needs one point and one turn.
F-22	RECOMMENDED	3	Declare the bounding dimensions in the description and keep them equal (within 1 cm) to the measured size of the geometry.
F-23	RECOMMENDED	3	Use true-to-life sizes: a work table is about 1.6 by 0.8 by 0.74 m, a chair about 0.5 by 0.55 by 0.85 m.
F-30	RECOMMENDED	3	Declare anchors in item coordinates (metres, relative to the pivot).
F-31	RECOMMENDED	3	Declare placement rules under `placement`: `mounting` (`floor`, `wall`, `ceiling`, `surface`), `againstWall`, `minWallDistance` (metres), `rotations` (allowed turns about the vertical axis, in degrees), `roomUses` (kinds of use it suits, free words matching the `Function` text of rooms), `avoid` (kinds of room it must not go in), and `mountHeight` for wall items.
F-32	RECOMMENDED	3	Declare clearance minimums around the item: `front`, `back`, `left`, `right`, `top` (metres of free volume needed).
F-33	RECOMMENDED	3	Keep items off door openings and off the floor directly in front of a door; a table in the doorway fails the doorway check (guide 1, B-42).
F-34	RECOMMENDED	3	Do not place items so that they cover a room's whole free floor (guide 1, B-33).
F-40	REQUIRED	3	Assume the profile's collision mode (`extensions.furnitureCollision.mode`).
F-41	RECOMMENDED	3	Place furniture with rotations in multiples of 90 degrees where you can; a box around a 45-degree table is larger than the table.
F-42	RECOMMENDED	3	Declare the footprint polygon and the collision box in the description (section 3.1), so a checker and a human can see what a consumer will do.
F-43	RECOMMENDED	3	Never make an item that must be walked under, through or between (an open pergola, a chair with gaps between its legs); in `box` mode it is a solid block.
F-44	RECOMMENDED	3	Avoid low obstacles up to `extensions.walkability.stepHeightMetres` that are not obviously steps (a floor box, a cable cover): a walker steps over them unnoticed.
F-50	REQUIRED	3	Stay within the profile's limits for a whole delivery: `limits.maxFileMegabytes`, `limits.maxVertices`, `limits.maxTriangles`.
F-51	RECOMMENDED	3	Keep each item small: at most `budgets.maxItemTriangles` triangles (default 2,000), far fewer for ordinary items.
F-52	RECOMMENDED	3	Do not deliver textures with an item.
F-53	RECOMMENDED	3	Use one or two surface colours on an ordinary item and up to four on a feature item; every distinct colour of every item adds a material.
F-60	RECOMMENDED	3	Describe variants in the description file.

Rule	Level	Guide	Statement
F-61	RECOMMENDED	3	Place repeated items as instances: each instance is an element of its own (own `GlobalId`, own `ObjectPlacement`), sharing geometry through `IfcMappedItem` and `IfcRepresentationMap`.
F-62	REQUIRED	3	Give each instance its own unique `GlobalId` (guide 1, B-70) and a level to belong to (guide 1, B-13).
F-62a	RECOMMENDED	3	Place every instance inside the room it serves, and do not let it extend through a wall or a floor.
F-63	RECOMMENDED	3	Record the item id and version on every instance (`ItemId`, `ItemVersion`, F-03) so that a revision of the library can be rolled out by replacing instances.
F-70	REQUIRED	3	Follow guide 1 exactly for light fixtures (section 11) and, when signboards are enabled, for signs (section 12).
F-71	RECOMMENDED	3	Make a fixture's housing and emissive part separate parts so that only the emissive part glows.
F-72	RECOMMENDED	3	Other fixtures (hand rails, handles, hooks, bathroom fittings, planters) are small elements with their own colour, in the classes of section 2.
X-01	REQUIRED	4	Use this layout.
X-02	RECOMMENDED	4	Use lower-case ASCII file names without blanks, forward slashes in every path, and keep paths under 200 characters.
X-10	REQUIRED	4	Put a `manifest.json` at the top of the package that matches the schema.
X-11	REQUIRED	4	Record the SHA-256 hash (64 lower-case hexadecimal characters) and size in bytes of every file, and include a `SHA256SUMS.txt` in the plain two-column form (`hash`, two blanks, `path`).
X-12	REQUIRED	4	List every file in the package, and only those, in the manifest.
X-13	RECOMMENDED	4	A building delivery has exactly one master IFC file (guide 1, B-03); other files are alternates.
X-14	REQUIRED	4	Keep the whole package within the profile's cap, `package.maxPackageMegabytes`, when it sets one.
X-20	REQUIRED	4	Give every IFC file, texture set and library item a `provenance` entry (what it is, where it came from: `original`, `procedural` with the script and seed, or a named source; who made it; the licence that applies) when the profile requires provenance (`package.requireProvenance`, default true).
X-21	REQUIRED	4	State a licence for every asset and a `licence.statement` for the package when the profile requires it (`package.requireLicence`, default true).
X-23	REQUIRED	4	When the delivery has terrain, list the patch and its sidecar under `terrain/` with their roles, give each a provenance entry, and record the source package name, version and hash and the patch hash in the manifest `terrain` section, equal to what the sidecar states and to what the check report of the master file found in it, and keep the site request (its building origin, azimuth and floor height must be the file's).
X-22	RECOMMENDED	4	Deliver only content you are free to hand over.
X-30	RECOMMENDED	4	Version the delivery, each texture set and each item with SemVer (`MAJOR.MINOR.PATCH`), and give every version a creation time.
X-31	RECOMMENDED	4	Choose the next version number by what changed.
X-32	RECOMMENDED	4	Write `notes/changes.md`: for each version, the ids added, changed and removed, in plain sentences.
X-50	REQUIRED	4	Never change a delivered version in place.

Rule	Level	Guide	Statement
X-40	REQUIRED	4	Run `check_ifc.py` on the exact master file you are sending, with the profile, and include its JSON report in the package.
X-41	RECOMMENDED	4	Deliver `tables/rooms.csv` with the columns `roomId`, `name`, `occupant`, `kind` (`room`, `circulation`, `observation`, `open`), `enclosed` (`yes` or `no`), `level`, `levelElevationMetres`, `floorAreaM2`, `clearHeightM`, `buildingId` and `layer`, and list every new `Occupant` string with the room-id prefix that goes with it and its kind, so a receiving party can add its rules before intake.
X-60	RECOMMENDED	4	When you send a revision, change only what the findings and your own notes name, bump the version (section 4), list what changed in `notes/changes.md`, and keep every unchanged identifier.
X-70	REQUIRED	4	Put a submitted delivery in `deliveries/<slug>/` with a `delivery.json` whose `delivery.id` equals the slug and whose `submission` object states the title, the submitter handle, the licence, the profile used, the tool versions and the attestation.
X-71	REQUIRED	4	Offer the delivery under a licence the profile lists (`submission.allowedLicences`; by default CC BY 4.0, or CC0 when the delivery says so).
X-72	REQUIRED	4	Include only files of the types the profile lists (`submission.allowedExtensions`; by default ifc, glb, png, jpg, json, md, csv and txt), with no program, script or archive and no executable file mode.
X-73	REQUIRED	4	Keep a delivery within the profile's limits: at most `submission.maxFiles` files (200), `submission.maxFileBytes` per file (25 MiB) and `submission.maxDeliveryBytes` in total (100 MiB).
X-74	REQUIRED	4	Use only relative, portable paths: no leading slash, no `..`, no backslash or drive letter, names of letters, digits, dot, hyphen and underscore that do not start with a dot, no reserved device names, no two names that differ only by case, and no path longer than `submission.maxPathLength`, in the folder and in every path the manifest and the checksum list name.
X-75	REQUIRED	4	Include no symbolic link and no special file, only plain files and folders.
X-76	REQUIRED	4	Have the submitter attest in `submission.attestation` that the work is their own or they may offer it, that they may license it under the stated licence, and that it holds no restricted third-party content.
X-77	REQUIRED	4	Make every file what its name says: PNG and JPEG images that are real images within `submission.maxImagePixels`, GLB files that are binary glTF 2 of the stated length, IFC files that are IFC text files, JSON that parses, text in UTF-8 without NUL bytes or active markup (script, iframe, object, embed).
X-78	RECOMMENDED	4	Add at least one preview picture under `previews/` (or `items/<id>/preview.png`) so a reviewer sees the work before opening a model.
X-79	REQUIRED	4	Put a proof manifest at `proofs/manifest.json` (format `proofs/1`) that lists every GLB, the GLB validation report and every render with its SHA-256, size and the tool and version that made it, and put nothing else in `proofs/`.
X-80	REQUIRED	4	Make every delivered GLB a valid binary glTF 2 file in metres with Y up, with no external reference, no unused buffer, image or material, finite and unit-length normals, tangents where a normal map is used, named scene-graph nodes, and the node, triangle and texture limits of the profile.
X-81	REQUIRED	4	Deliver a GLB of the whole master model and one of every item, covering, fixture, hardware piece, door, window and texture set, each made from the delivered IFC files (their SHA-256 is recorded), together with a validation report that passes and names every GLB by its SHA-256.

Rule	Level	Guide	Statement
X-82	REQUIRED	4	Include renders that cover every level (a plan), every room kind present (the profile's number of interior views), the outside of the building, and every item, covering, fixture, hardware piece, door, window and texture set (a contact sheet).
X-83	REQUIRED	4	Make every render from a GLB of the delivery and record in the manifest the SHA-256 of that GLB, of the picture itself, its pixel size, the camera and the lighting; each recorded GLB hash must be the hash of the delivered GLB file.
X-84	REQUIRED	4	Keep every render within the profile's pixel and file-size limits (`proofs.minRenderWidth`, `proofs.maxRenderPixels`, `proofs.maxRenderBytes`) and the whole `proofs/` folder within `proofs.maxProofBytes`.
X-85	REQUIRED	4	Keep personal data and local paths out of images and GLB files: renders hold only the image chunks the profile allows, no image holds EXIF, XMP, a colour profile with text or text that names a person, a folder, a host or an address, and no GLB holds a path, an address, an e-mail address or a generator other than the set's.
X-86	REQUIRED	4	Make the proofs reproducible: rebuilding the GLB files from the delivered IFC files must give the same bytes, and re-rendering the delivered GLB files must give pictures that differ from the delivered ones by no more than `proofs.renderTolerance`; where the toolchain is not available the check says so and is skipped, not passed.